

FAKE IMAGE DETECTION SYSTEM FEEDBACK

A PROJECT REPORT

In the partial fulfilment of the requirements for the award of

MASTER OF COMPUTER APPLICATIONS

Submitted by

BODDU VASAVI

(REGD. NO: 324228860014)

Under the Esteemed Guidance of

Mrs. B. V. P. LATHA

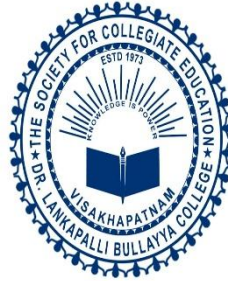
Assistant Professor

INTERNAL GUIDE

Sri. M. A. PRASAD

Sr. Assistant Professor

HEAD OF THE DEPARTMENT



DEPARTMENT OF COMPUTER SCIENCE

DR. LANKAPALLI BULLAYYA P.G COLLEGE

(AFFILIATED TO ANDHRA UNIVERSITY)

VISAKHAPATNAM-530013

(2024-2026)

ACKNOWLEDGEMENT

At the very outset, I express my gratitude to the Almighty Lord for showering His grace and blessings upon me to complete this project. Although my name appears on the cover of this book, many people have contributed in one form or another to this project's development. We could not have done this project without the assistance and support of each of the following individuals, and we thank you all.

We express our gratitude towards our Principal, **Prof. G.S.K. CHAKRAVARTY**, Dean of Dr. Lankapalli Bullayya P.G. College, for providing the necessary departmental facilities in spite of his busy schedule and for guiding us in every possible way.

We are indeed very grateful to our beloved Head of the Department, **Sri. M.A. PRASAD**, Head of the Department of Computer Science, Dr. Lankapalli Bullayya P.G. College, for his constant willingness to share his valuable time and for inspiring us through his suggestions.

We express our deep gratitude to our project guide, **MRS. B. V. P. LATHA**, Assistant Professor in Computer Science, Department of Computer Science, Dr. Lankapalli Bullayya P.G. College, for rendering guidance and valuable advice at all times. She has been a perennial source of inspiration and motivation throughout this project.

We sincerely thank all the staff members of the department for giving us their support during all stages of the project work and its completion. In all humility and reverence, we express our profound sense of gratitude to all elders and professors who have willingly spared their time, experience, and knowledge to guide us in our project.

BODDU VASAVI
(REG. NO: 324228860014)

DECLARATION

I hereby declare that this project report entitled **“FAKE IMAGE DETECTION SYSTEM”** is the result of original work done by me to the best of my knowledge. A similar work has not been submitted previously to any other university or published before. This project is submitted in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications.

BODDU VASAVI

(REG. NO:324228860014)

ABSTRACT

The rapid growth of digital technologies and artificial intelligence has significantly increased the creation and sharing of digital images through social media, online platforms, and communication applications. However, advanced image editing software and AI-based image generation tools have made it easy to manipulate or create highly realistic fake images. Such images are widely used in spreading misinformation, fake news, cybercrime, identity theft, financial fraud, and political propaganda, making image authenticity verification an essential requirement in digital forensics, cybersecurity, journalism, and law enforcement.

The Fake Image Detection System presented in this project provides an intelligent and automated solution for detecting manipulated and AI-generated images using a combination of digital image forensic techniques and deep learning. Unlike traditional methods that rely on a single detection approach, the proposed system integrates Metadata Analysis, Error Level Analysis (ELA), ORB (Oriented FAST and Rotated BRIEF) Copy-Move Detection, and a Convolutional Neural Network (CNN) to improve detection accuracy and reliability. Each technique examines different characteristics of an image, enabling comprehensive analysis before generating the final prediction.

The proposed system offers several advantages, including improved detection accuracy, faster image verification, transparent analysis, and reduced dependence on manual inspection. It can be effectively applied in digital forensic investigations, social media content verification, journalism, cybersecurity, legal evidence verification, academic research, and online content moderation. Its modular architecture also supports future enhancements such as deepfake detection, video forgery detection, cloud deployment, mobile applications, and advanced deep learning models.

The Fake Image Detection System demonstrates the effective integration of digital image forensic techniques and artificial intelligence to identify manipulated digital images. By combining multiple detection methods with deep learning, the system provides a reliable and efficient solution for improving digital trust, preventing the spread of fake visual information, and supporting the advancement of digital image authentication technologies.

INDEX

CONTENT	PAGE NO
1.Introduction	1-15
1.1 Introduction to the Project.	1- 3
1.1.1 Scope.	4- 5
1.1.2 Problem statement.	5 - 9
1.2 Existing System	9 - 12
1.3 Proposed System including modules.	12 - 15
2. Requirement Analysis	16 - 28
2.1 Feasibility study	18
2.1.1. Technical Feasibility	19
2.1.2. Economic Feasibility	20
2.1.2.1. Cost Benefit Analysis	20
2.1.3. Operational Feasibility	21
2.2. Software Requirement Specification	21
2.2.1. Functional Requirements	22
2.2.1.1. Input	23
2.2.1.2. Output	23
2.2.1.3. Computational	24
2.2.1.4. Storage (Database Requirements)	24
2.2.2. Non-Functional Requirements	25
2.2.2.1. Quality Metrics	25
2.2.2.2. Reliability	25
2.2.2.3. Usability	26
2.2.2.4. Response Time	26
2.2.2.5. Throughput	26
2.2.2.6. Availability	27
2.2.2.7. Maintainability	27
2.2.3 Environment & Technology Requirements	27
2.2.3.1. Hardware Requirements	28
2.2.3.2. Software Requirements	28
3. Design	30 - 48
3.1. System Design	30 - 31
3.1.1. Introduction to UML.	31 - 32
3.1.2. UML Diagrams.	32
3.1.2.1. Scenarios.	32

3.1.2.2. Use-Case Diagrams.	33
3.1.2.3. Class Diagrams.	34
3.1.2.4. Object Diagrams.	35
3.1.2.5. Sequence Diagrams (or Collaboration Diagrams).	36
3.1.2.6. Activity Diagrams.	37
3.1.2.7. State Diagrams.	38
3.1.2.8. Component Diagrams. (Optional)	39
3.1.2.9. Deployment Diagrams (Optional)	40
3.2. Interface Design (Input forms)	41 - 44
3.3. Database Design	45
3.3.1. Introduction to Backend.	46
3.3.2. Normalization regarding the project tables.	46 - 47
3.3.3. Table Description.	47 - 48
4. Coding	49 - 52
4.1. Sample Code (3-4 pages).	49 - 52
5. Testing	53 - 57
5.1. Introduction to Testing.	53
5.2. Test Cases (module- wise)	53 - 57
6. Screenshots	58 - 61
6.1. Input Screens (Forms).	58 - 59
6.2. Output Screens (Reports).	59 - 61
7. Conclusion	62
8. Bibliography	63

1.INTRODUCTION

1.1 Introduction to the Project:

The rapid advancement of information technology has transformed the way digital information is created, processed, stored, and shared across the world. Among the various forms of digital media, digital images have become one of the most widely used methods of communication because they provide visual information that is easy to understand and interpret. With the increasing use of smartphones, digital cameras, surveillance systems, and social media platforms, billions of digital images are generated and shared every day. These images are extensively used in journalism, education, healthcare, banking, legal investigations, scientific research, e-commerce, entertainment, and social networking. The increasing dependence on digital images has made image authenticity an important aspect of modern digital communication.

Although digital images provide numerous advantages, they also introduce several security challenges. Modern image editing software such as Adobe Photoshop, GIMP, Canva, and many AI-powered editing applications allow users to modify digital images with remarkable precision. Editing operations such as cropping, resizing, brightness adjustment, contrast enhancement, background replacement, object removal, image splicing, and facial modifications can be performed within a few minutes. While these editing tools are useful for professional photography, graphic design, advertising, and educational purposes, they can also be misused for creating fake or manipulated images. Such manipulated images are often difficult to distinguish from genuine photographs through manual observation alone.

The emergence of Artificial Intelligence has further accelerated the development of realistic image generation technologies. AI models such as Generative Adversarial Networks (GANs) and diffusion models can generate high-quality synthetic images that closely resemble real photographs. These technologies have numerous beneficial applications in animation, digital art, game development, architecture, medical imaging, and education. However, they also create opportunities for misuse by enabling the creation of fake human faces, manipulated photographs, forged identity documents, misleading social media posts, and fabricated news content. The increasing availability of AI-generated images has made traditional image verification methods less effective, thereby increasing the demand for intelligent fake image detection systems.

Fake and manipulated images can have serious consequences in various sectors. In journalism, fake images may spread false news and damage the credibility of media organizations. In legal investigations, manipulated photographs may affect the authenticity of digital evidence presented in courts. Financial institutions face risks when forged documents containing manipulated photographs are used for fraudulent activities. Social media platforms frequently encounter fake images that mislead users, create unnecessary panic, and influence public opinion. Cybercriminals also use manipulated images for phishing attacks, identity theft, and online scams. Therefore, verifying the authenticity of digital images has become an important requirement in ensuring digital trust and information security.

Digital Image Forensics is a specialized branch of computer science that focuses on identifying modifications made to digital images. It uses various computational techniques to analyse compression

artifacts, metadata information, duplicated image regions, colour inconsistencies, texture variations, and statistical image properties. These forensic techniques help investigators determine whether an image has been manipulated after its original creation. As image editing techniques become more advanced, digital forensic systems must also evolve to accurately identify sophisticated image manipulations.

Machine Learning and Deep Learning have significantly improved the field of digital image analysis. Deep learning models, particularly Convolutional Neural Networks (CNNs), automatically learn complex image features from large datasets without requiring manual feature engineering. CNN models have demonstrated remarkable performance in image classification, facial recognition, medical diagnosis, satellite image analysis, and digital image forensics. Their capability to identify subtle pixel-level variations makes them highly suitable for fake image detection applications.

The Fake Image Detection System developed in this project is designed to provide an intelligent, efficient, and user-friendly solution for identifying manipulated digital images. The application combines multiple image forensic techniques with deep learning to improve detection accuracy. Instead of relying on only one detection method, the proposed system integrates Metadata Analysis, Error Level Analysis (ELA), ORB Copy-Move Detection, and a Convolutional Neural Network (CNN) into a single application. Each technique performs a specific role in analysing the uploaded image and contributes to the final prediction. The project has been developed using the Python programming language, which provides extensive support for machine learning, image processing, and web application development. The user interface is created using the Streamlit framework, allowing users to upload images through a simple web-based application. Various Python libraries including TensorFlow, OpenCV, NumPy, and Pillow (PIL) are used for image preprocessing, feature extraction, forensic analysis, and deep learning classification. The application accepts an uploaded image, processes it through multiple forensic stages, and finally predicts whether the image is Real or Fake.

The first stage of the system performs Metadata Analysis. Metadata contains hidden information about the image, including camera model, image dimensions, creation date, GPS location, and other technical details. Genuine photographs captured using digital cameras or smartphones generally contain complete metadata, whereas AI-generated images or manipulated images often have missing or modified metadata. Therefore, metadata analysis provides an initial indication of image authenticity.

The second stage performs Error Level Analysis (ELA). ELA is one of the most widely used digital forensic techniques for detecting image tampering. It works by identifying variations in JPEG compression levels across different regions of an image. When a JPEG image is edited and saved multiple times, manipulated regions often exhibit different compression characteristics compared to the original portions of the image. ELA highlights these differences, making edited regions easier to identify.

The third stage utilizes ORB (Oriented FAST and Rotated BRIEF) Copy-Move Detection. Copy-move forgery is a common image manipulation technique where a portion of an image is copied and pasted elsewhere within the same image to conceal or duplicate objects. The ORB algorithm extracts key points and

feature descriptors from the image and compares them to identify duplicated regions. If similar feature patterns are detected in different locations within the image, the application considers them as potential indicators of copy-move forgery.

Finally, the processed image is analysed using a trained Convolutional Neural Network (CNN) model. The CNN automatically extracts hierarchical visual features and classifies the image as Real or Fake based on patterns learned during training. Unlike traditional machine learning methods, CNN does not require manual feature extraction, making it more efficient and accurate in identifying manipulated images. The final prediction is displayed along with the confidence score and technical analysis, enabling users to understand the basis of the system's decision.

One of the major strengths of the proposed Fake Image Detection System is its multi-layered verification approach. Instead of depending on a single forensic technique, the system combines metadata analysis, image compression analysis, feature matching, and deep learning classification to improve reliability. This integration reduces false predictions and enhances the overall performance of the application. The system is capable of assisting journalists, law enforcement agencies, cybersecurity professionals, educational institutions, researchers, and social media users in verifying the authenticity of digital images before accepting them as genuine.

The proposed project demonstrates the practical application of Artificial Intelligence, Image Processing, Digital Image Forensics, and Deep Learning in solving a real-world cybersecurity problem. It provides a simple, efficient, and scalable solution that can be further enhanced with advanced AI models, cloud deployment, video forgery detection, and real-time image verification techniques. By integrating multiple forensic approaches into a single platform, the Fake Image Detection System contributes to improving digital trust and reducing the spread of manipulated visual information across various online platforms

1.1.1 Scope:

The scope of the Fake Image Detection System is to provide an intelligent, efficient, and user-friendly platform for identifying whether a digital image is genuine or manipulated. As digital images are increasingly used in communication, journalism, education, business, legal investigations, and social media, verifying their authenticity has become an important requirement. The proposed system addresses this requirement by integrating image processing techniques with artificial intelligence to analyse uploaded images and determine their authenticity.

The project primarily focuses on detecting manipulated images using multiple forensic analysis techniques instead of relying on a single detection method. The application performs Metadata Analysis, Error Level Analysis (ELA), ORB Copy-Move Detection, and Convolutional Neural Network (CNN) based classification to evaluate the uploaded image. Each technique contributes unique information that helps improve the overall prediction accuracy. The combination of these techniques increases the reliability of the system and reduces the possibility of incorrect classification.

The proposed system is designed to process commonly used image formats such as JPG, JPEG, and PNG. The user can upload an image through the Stream lit web interface without requiring any technical knowledge. Once the image is uploaded, the application automatically performs preprocessing, metadata extraction, ELA generation, ORB feature analysis, and CNN prediction before displaying the final result. This makes the system simple, efficient, and suitable for both technical and non-technical users.

The project has wide applications in several domains. Journalists can use the system to verify the authenticity of images before publishing news articles. Law enforcement agencies can utilize it for digital evidence verification during criminal investigations. Cybersecurity professionals can analyse suspicious images involved in cybercrime, phishing attacks, and identity fraud. Educational institutions can use the application as a learning tool for digital image forensics and artificial intelligence. Researchers can further extend the project by incorporating advanced deep learning models and additional forensic techniques. The Fake Image Detection System also provides a strong foundation for future development. The existing application can be enhanced by integrating Deepfake image detection, video forgery detection, cloud deployment, mobile application support, real-time image authentication, and transformer-based deep learning models. Additional forensic techniques such as image noise analysis, illumination consistency analysis, frequency domain analysis, and GAN artifact detection can also be incorporated to improve detection performance. Therefore, the scope of the project extends beyond the current implementation and provides opportunities for continuous research and development in digital image forensics.

Although the proposed system achieves satisfactory performance for detecting manipulated images, certain limitations remain. The prediction accuracy depends on the quality and diversity of the training dataset. Highly sophisticated AI-generated images may still require more advanced deep learning architectures and larger datasets for accurate detection. Nevertheless, the project successfully demonstrates the practical integration of artificial intelligence and digital image forensic techniques for solving a real-world problem and provides an effective platform for future enhancements.

Overall, the scope of the Fake Image Detection System extends across multiple industries and application domains where image authenticity is important. By combining intelligent image analysis techniques with an easy-to-use interface, the project contributes toward improving digital trust, reducing the spread of fake visual information, and supporting secure digital communication in modern society.

1.1.2 Problem Statement:

The rapid advancement of digital image editing software and Artificial Intelligence has made it extremely easy to create realistic manipulated images. Modern editing tools allow users to alter images by removing or adding objects, replacing backgrounds, modifying facial features, adjusting lighting conditions, and performing copy-move forgery without leaving obvious visual evidence. Furthermore, AI-based image generation technologies are capable of creating entirely synthetic images that closely resemble genuine photographs. As a result, distinguishing between authentic and manipulated images has become increasingly difficult through manual observation.

The widespread availability of fake images has created significant challenges across various sectors. Social media platforms are flooded with manipulated images that spread misinformation, fake news, political propaganda, and misleading advertisements. Fake images can damage the reputation of individuals, organizations, and governments while creating unnecessary panic among the public. In journalism, publishing manipulated images may result in the spread of false information and reduce public confidence in news organizations. Similarly, forged images are frequently used in identity theft, online fraud, cybercrime, insurance scams, and document forgery, leading to financial losses and security threats.

Traditional methods of image verification depend heavily on human expertise and manual visual inspection. However, manual analysis is time-consuming, subjective, and often unreliable when dealing with professionally edited or AI-generated images. Human observers may overlook subtle image manipulations that are invisible to the naked eye. Moreover, the enormous volume of digital images uploaded every day makes manual verification practically impossible. Therefore, there is a growing need for automated systems capable of accurately detecting manipulated images with minimal human intervention.

Many existing fake image detection systems rely on only a single forensic technique such as metadata analysis or image compression analysis. Although these techniques perform well under specific conditions, they often fail when images are manipulated using advanced editing methods or AI-based generation tools. A single detection technique cannot effectively identify all types of image forgeries because different manipulation methods leave different forensic traces. Consequently, existing systems may produce inaccurate predictions or fail to detect sophisticated image manipulations.

The primary problem addressed in this project is the absence of a simple, reliable, and intelligent system capable of performing comprehensive image authentication using multiple forensic analysis techniques. The proposed Fake Image Detection System solves this problem by integrating Metadata Analysis, Error Level Analysis (ELA), ORB Copy-Move Detection, and Convolutional Neural Network (CNN) based classification into a single application. This multi-layered verification approach improves detection accuracy and provides more reliable results compared to systems that depend on only one detection method.

Another important problem addressed by the project is accessibility. Many existing forensic tools require specialized technical knowledge and are difficult for ordinary users to operate. The proposed system overcomes this limitation by providing a simple Streamlit-based web interface that allows users to upload an image and obtain the analysis results with minimal effort. This improves usability and enables both technical and non-technical users to verify image authenticity.

The project also addresses the growing need for trustworthy digital information. As digital communication continues to expand, ensuring the authenticity of visual content has become essential for maintaining public trust, supporting legal investigations, preventing cybercrime, and reducing the spread of misinformation. By providing an intelligent and automated image verification system, the Fake Image Detection System contributes to strengthening digital security and improving confidence in digital media.

In summary, the problem addressed by this project is the increasing difficulty of distinguishing genuine images from manipulated or AI-generated images using traditional methods. The proposed solution combines multiple image forensic techniques with deep learning to develop an efficient, accurate, and user-friendly system capable of detecting fake images. The project therefore provides a practical solution to one of the major challenges faced in digital image forensics and cybersecurity.

Although digital images provide numerous advantages, they also introduce several security challenges. Modern image editing software such as Adobe Photoshop, GIMP, Canva, and many AI-powered editing applications allow users to modify digital images with remarkable precision. Editing operations such as cropping, resizing, brightness adjustment, contrast enhancement, background replacement, object removal, image splicing, and facial modifications can be performed within a few minutes. While these editing tools are useful for professional photography, graphic design, advertising, and educational purposes, they can also be misused for creating fake or manipulated images. Such manipulated images are often difficult to distinguish from genuine photographs through manual observation alone.

The emergence of Artificial Intelligence has further accelerated the development of realistic image generation technologies. AI models such as Generative Adversarial Networks (GANs) and diffusion models can generate high-quality synthetic images that closely resemble real photographs. These technologies have numerous beneficial applications in animation, digital art, game development, architecture, medical imaging, and education. However, they also create opportunities for misuse by enabling the creation of fake human faces, manipulated photographs, forged identity documents, misleading social media posts, and fabricated news content. The increasing availability of AI-generated images has made traditional image verification methods less effective, thereby increasing the demand for intelligent fake image detection systems.

Fake and manipulated images can have serious consequences in various sectors. In journalism, fake images may spread false news and damage the credibility of media organizations. In legal investigations, manipulated photographs may affect the authenticity of digital evidence presented in courts. Financial institutions face risks when forged documents containing manipulated photographs are used for fraudulent activities. Social media platforms frequently encounter fake images that mislead users, create unnecessary panic, and influence public opinion. Cybercriminals also use manipulated images for phishing attacks, identity theft, and online scams. Therefore, verifying the authenticity of digital images has become an important requirement in ensuring digital trust and information security.

Digital Image Forensics is a specialized branch of computer science that focuses on identifying modifications made to digital images. It uses various computational techniques to analyse compression artifacts, metadata information, duplicated image regions, colour inconsistencies, texture variations, and statistical image properties. These forensic techniques help investigators determine whether an image has been manipulated after its original creation. As image editing techniques become more advanced, digital forensic systems must also evolve to accurately identify sophisticated image manipulations.

Machine Learning and Deep Learning have significantly improved the field of digital image analysis. Deep learning models, particularly Convolutional Neural Networks (CNNs), automatically learn complex image features from large datasets without requiring manual feature engineering. CNN models have demonstrated remarkable performance in image classification, facial recognition, medical diagnosis, satellite image analysis, and digital image forensics. Their capability to identify subtle pixel-level variations makes them highly suitable for fake image detection applications.

The Fake Image Detection System developed in this project is designed to provide an intelligent, efficient, and user-friendly solution for identifying manipulated digital images. The application combines multiple image forensic techniques with deep learning to improve detection accuracy. Instead of relying on only one detection method, the proposed system integrates Metadata Analysis, Error Level Analysis (ELA), ORB Copy-Move Detection, and a Convolutional Neural Network (CNN) into a single application. Each technique performs a specific role in analysing the uploaded image and contributes to the final prediction. The project has been developed using the Python programming language, which provides extensive support for machine learning, image processing, and web application development. The user interface is created using the Streamlit framework, allowing users to upload images through a simple web-based application. Various Python libraries including TensorFlow, OpenCV, NumPy, and Pillow (PIL) are used for image preprocessing, feature extraction, forensic analysis, and deep learning classification. The application accepts an uploaded image, processes it through multiple forensic stages, and finally predicts whether the image is Real or Fake.

The first stage of the system performs Metadata Analysis. Metadata contains hidden information about the image, including camera model, image dimensions, creation date, GPS location, and other technical details. Genuine photographs captured using digital cameras or smartphones generally contain complete metadata, whereas AI-generated images or manipulated images often have missing or modified metadata. Therefore, metadata analysis provides an initial indication of image authenticity.

The second stage performs Error Level Analysis (ELA). ELA is one of the most widely used digital forensic techniques for detecting image tampering. It works by identifying variations in JPEG compression levels across different regions of an image. When a JPEG image is edited and saved multiple times, manipulated regions often exhibit different compression characteristics compared to the original portions of the image. ELA highlights these differences, making edited regions easier to identify.

The third stage utilizes ORB (Oriented FAST and Rotated BRIEF) Copy-Move Detection. Copy-move forgery is a common image manipulation technique where a portion of an image is copied and pasted elsewhere within the same image to conceal or duplicate objects. The ORB algorithm extracts key points and feature descriptors from the image and compares them to identify duplicated regions. If similar feature patterns are detected in different locations within the image, the application considers them as potential indicators of copy-move forgery.

Finally, the processed image is analysed using a trained Convolutional Neural Network (CNN) model. The CNN automatically extracts hierarchical visual features and classifies the image as Real or Fake based on patterns learned during training. Unlike traditional machine learning methods, CNN does not require manual feature extraction, making it more efficient and accurate in identifying manipulated images. The final prediction is displayed along with the confidence score and technical analysis, enabling users to understand the basis of the system's decision.

One of the major strengths of the proposed Fake Image Detection System is its multi-layered verification approach. Instead of depending on a single forensic technique, the system combines metadata analysis, image compression analysis, feature matching, and deep learning classification to improve reliability. This integration reduces false predictions and enhances the overall performance of the application. The system is capable of assisting journalists, law enforcement agencies, cybersecurity professionals, educational institutions, researchers, and social media users in verifying the authenticity of digital images before accepting them as genuine.

The proposed project demonstrates the practical application of Artificial Intelligence, Image Processing, Digital Image Forensics, and Deep Learning in solving a real-world cybersecurity problem. It provides a simple, efficient, and scalable solution that can be improving digital trust and reducing the spread of manipulated visual information across various online platforms.

1.2 Existing System:

The existing systems used for detecting fake or manipulated images mainly depend on traditional image forensic techniques or manual image verification methods. In most cases, these systems analyse only one aspect of the image, such as metadata, compression artifacts, or pixel information, to determine whether an image has been manipulated. Although these methods provide useful information, they often fail to identify sophisticated image manipulations created using modern image editing software or Artificial Intelligence techniques.

One of the most common approaches followed by existing systems is manual image verification. In this method, forensic experts or experienced users visually examine the image to identify any suspicious modifications. They observe the image carefully for irregular shadows, inconsistent lighting, duplicated objects, unnatural edges, or colour mismatches. While this method may successfully detect obvious image manipulations, it becomes highly ineffective when images are professionally edited. Modern editing software enables seamless image modifications that are almost impossible to detect through human observation alone. Therefore, manual verification is time-consuming, subjective, and unsuitable for processing the large number of digital images generated every day.

Another widely used technique in existing systems is metadata analysis. Metadata contains hidden information embedded within a digital image, including camera manufacturer, camera model, date and time of image capture, image resolution, GPS location, software used for editing, and other technical properties. Existing forensic tools examine this metadata to determine whether the image appears genuine. However,

metadata can be easily removed, modified, or completely deleted using freely available editing tools. AI-generated images and screenshots often contain little or no metadata, making this technique insufficient when used alone.

Some existing image verification systems depend on Error Level Analysis (ELA) to identify manipulated regions. ELA highlights variations in JPEG compression across different areas of the image. Images that have been edited and saved multiple times usually exhibit different compression characteristics in manipulated regions. Although ELA is an effective forensic technique for JPEG images, it has several limitations. It does not perform well for images stored in PNG or other lossless formats. Similarly, images that have undergone multiple compressions or extensive post-processing may produce misleading ELA results, thereby reducing detection accuracy.

Several traditional systems also utilize feature-based forgery detection algorithms such as SIFT (Scale Invariant Feature Transform), SURF (Speeded Up Robust Features), and ORB (Oriented FAST and Rotated BRIEF). These algorithms identify duplicated regions within an image by comparing feature descriptors extracted from different image locations. Such techniques are particularly useful for detecting copy-move forgery, where a portion of the image is copied and pasted elsewhere within the same image. However, these methods are generally limited to specific manipulation types and cannot effectively detect AI-generated images or image splicing.

In recent years, researchers have developed machine learning-based image classification systems capable of detecting manipulated images using statistical image features. These systems require manually extracted features before classification and often depend heavily on the quality of feature engineering. The performance of these systems decreases significantly when images contain complex editing operations or previously unseen manipulation techniques.

Deep learning-based approaches have improved fake image detection considerably by automatically learning image features from training datasets. Convolutional Neural Networks (CNNs) have achieved higher accuracy compared to traditional machine learning algorithms because they automatically extract complex visual features. However, many existing CNN-based systems depend solely on deep learning predictions without incorporating additional forensic analysis techniques such as metadata verification or copy-move detection. Consequently, the overall reliability of these systems may decrease when images differ significantly from the training dataset.

Another limitation of existing systems is the lack of an integrated platform that combines multiple forensic techniques within a single application. Most available tools perform only one specific type of analysis, requiring investigators to use several independent software applications for complete image verification. This process is time-consuming, inconvenient, and often requires specialized technical expertise. Furthermore, many commercial forensic software packages are expensive and inaccessible to students, researchers, and small organizations.

Most existing applications also lack a simple and interactive user interface. Many forensic tools require users to possess advanced knowledge of digital image processing, making them unsuitable for ordinary users. The absence of user-friendly interfaces limits the practical adoption of these systems in educational institutions, small organizations, and general public applications.

1.3 Proposed System:

The Fake Image Detection System is an intelligent web-based application developed to identify whether a digital image is genuine or manipulated. The proposed system has been designed to overcome the limitations of existing image verification techniques by integrating multiple digital image forensic methods with Artificial Intelligence. Instead of relying on a single detection algorithm, the application combines Metadata Analysis, Error Level Analysis (ELA), ORB (Oriented FAST and Rotated BRIEF) Copy-Move Detection, and Convolutional Neural Network (CNN) based image classification. This multi-layered approach significantly improves the reliability and accuracy of image authentication.

The proposed system has been implemented using the Python programming language, which is widely used for Artificial Intelligence, Machine Learning, Image Processing, and Web Application Development. Python offers numerous libraries that simplify the implementation of digital image forensic algorithms. The graphical user interface of the application is developed using the Streamlit framework, enabling users to upload an image and receive the prediction results through a simple and interactive web interface.

The workflow of the proposed system begins when the user uploads a digital image through the Streamlit application. The uploaded image is first validated to ensure that it is in a supported format such as JPG, JPEG, or PNG. Once the image is accepted, it is passed through several analysis stages. Each stage performs a specific forensic operation and generates valuable information that contributes to the final prediction. The processed information is finally analysed by the Convolutional Neural Network model, which predicts whether the uploaded image is Real or Fake.

One of the important features of the proposed system is Metadata Analysis. Every digital image contains hidden information known as metadata or EXIF data. This information may include camera model, manufacturer, image dimensions, creation date, GPS coordinates, exposure settings, and software details. Genuine photographs usually retain their original metadata, whereas manipulated images or AI-generated images often have missing or modified metadata. The application extracts the available metadata and presents it to the user as part of the technical analysis. Although metadata alone cannot determine image authenticity, it provides useful supporting evidence for forensic investigation.

The second stage of the system performs Error Level Analysis (ELA). ELA is an effective forensic technique used to identify image tampering by analysing JPEG compression differences. The uploaded image is recompressed using a predefined JPEG quality level and then compared with the original image. The differences between the two images are amplified to generate the ELA output. Regions showing unusually high error levels indicate possible image modifications. The generated ELA image is displayed to the user for better understanding of suspicious regions within the uploaded image.

The third stage utilizes the ORB Copy-Move Detection Algorithm. Copy-move forgery is one of the most common forms of image manipulation where a portion of an image is copied and pasted elsewhere within the same image to hide or duplicate objects. The ORB algorithm detects feature points and generates descriptors for different regions of the image. These descriptors are compared using the Brute Force Matcher to identify duplicated regions. If a significant number of matching key points are detected, the application considers it as an indication of possible copy-move forgery. This module enhances the ability of the system to detect image manipulation that may not be visible through ELA alone.

The final and most important stage is the Convolutional Neural Network (CNN) Prediction Module. The processed image generated by the ELA module is resized to the required dimensions and supplied to the trained CNN model. The CNN automatically extracts important visual features from the image using convolutional layers and performs binary classification using fully connected dense layers. The model predicts whether the uploaded image belongs to the Real or Fake category and calculates the confidence score associated with the prediction. The confidence score provides additional information regarding the certainty of the classification result.

Unlike traditional systems that depend on only one analysis technique, the proposed application combines the outputs of multiple forensic methods before generating the final result. This integrated approach improves detection accuracy because different manipulation techniques leave different forensic traces. Metadata analysis identifies missing or altered image information, ELA detects compression inconsistencies, ORB identifies duplicated regions, and CNN performs intelligent image classification. Together, these techniques provide a comprehensive image authentication mechanism.

The application also emphasizes simplicity and ease of use. Users do not require specialized knowledge of digital image forensics or artificial intelligence. They only need to upload an image through the web interface, after which the application automatically performs all analysis stages and displays the results. The final output includes the uploaded image, metadata information, ELA output, ORB analysis result, CNN prediction, confidence score, and technical interpretation. This makes the application suitable for students, journalists, researchers, cybersecurity professionals, and general users.

The proposed system has been designed with a modular architecture, making it easy to maintain and upgrade. New forensic techniques or advanced deep learning models can be integrated into the existing framework without significant modifications. This flexibility enables future improvements such as Deepfake detection, video forgery analysis, cloud-based deployment, mobile application development, and transformer-based image classification models.

The Fake Image Detection System provides several advantages over existing methods. It combines multiple forensic techniques, improves prediction accuracy, reduces false classifications, offers an interactive user interface, supports rapid image analysis, and provides detailed technical explanations for every prediction. These features make the proposed system a practical solution for digital image authentication in various real-

world applications including digital forensics, social media monitoring, cybercrime investigation, journalism, legal evidence verification, academic research, and online content moderation.

Overall, the proposed system successfully integrates Digital Image Processing, Artificial Intelligence, Machine Learning, Deep Learning, and Digital Image Forensics into a single intelligent application. The project demonstrates how multiple forensic techniques can be combined to develop an efficient, reliable, and user-friendly image authentication system capable of addressing the growing challenges associated with fake and manipulated digital images.

1.3.1 Modules:

The proposed Fake Image Detection System consists of the following functional modules:

1. Image Upload Module:

This module provides the interface for users to upload images in supported formats (JPG, JPEG, PNG). It validates the uploaded file and forwards it to the analysis modules.

2. Metadata Analysis Module:

This module extracts EXIF metadata from the uploaded image, including camera information, creation date, resolution, and software details. The extracted metadata assists in preliminary forensic analysis.

3. Error Level Analysis (ELA) Module:

The uploaded image is recompressed and compared with the original image to identify differences in JPEG compression. Suspicious regions are highlighted in the generated ELA image.

4. ORB Copy-Move Detection Module:

This module uses the ORB algorithm to detect duplicated image regions by matching key points and feature descriptors. It helps identify copy-move forgery within the image.

5. CNN Prediction Module:

The processed image is supplied to the trained Convolutional Neural Network model, which classifies the image as Real or Fake and calculates the prediction confidence.

6. Result Display Module:

This module displays the uploaded image, prediction result, confidence score, metadata information, ELA image, ORB analysis, and other technical details through the Streamlit interface.

7. Technical Analysis Module:

This module combines the outputs from all forensic techniques and presents a detailed technical interpretation of the image analysis, enabling users to understand the basis of the final prediction.

2. REQUIREMENTS ANALYSIS

Requirement Analysis is one of the most important phases in the Software Development Life Cycle (SDLC). It is the process of identifying, analysing, documenting, and validating the requirements of a software system before the actual development begins. This phase acts as a bridge between the users and developers by ensuring that the final software product satisfies the needs and expectations of the users. Proper requirement analysis minimizes development risks, reduces implementation errors, improves software quality, and ensures that the developed system meets its intended objectives. A well-defined requirement analysis also helps developers estimate the project cost, development time, required resources, and overall project feasibility.

For the Fake Image Detection System, requirement analysis plays a significant role because the application integrates Artificial Intelligence, Digital Image Processing, and Digital Image Forensics into a single platform. Since the project performs multiple image analysis operations, it is necessary to identify all functional and non-functional requirements before implementation. The requirement analysis phase helps determine the software tools, programming languages, libraries, hardware resources, datasets, and user requirements needed for successful project development.

The primary objective of the Fake Image Detection System is to develop an intelligent application capable of determining whether an uploaded digital image is genuine or manipulated. The system should provide an easy-to-use interface where users can upload images without requiring technical knowledge. After receiving the uploaded image, the application should automatically perform Metadata Analysis, Error Level Analysis (ELA), ORB Copy-Move Detection, and Convolutional Neural Network (CNN) based classification before displaying the prediction result. Therefore, identifying the functional behaviour of each module is an essential part of the requirement analysis process.

During the requirement analysis phase, the expected users of the application were identified. The proposed system can be utilized by journalists, digital forensic investigators, cybersecurity professionals, students, researchers, educational institutions, social media content moderators, and law enforcement agencies. Since the system is intended for a wide range of users, the application must provide a simple graphical interface that allows image verification through a few simple steps. User friendliness and ease of operation therefore become important requirements of the project.

Another important aspect considered during requirement analysis is the identification of software dependencies. The application is developed using the Python programming language because of its extensive support for Artificial Intelligence and Image Processing. Streamlit is selected for developing the graphical user interface because it allows rapid development of web applications without requiring HTML or JavaScript programming. TensorFlow and Keras are used for developing and executing the Convolutional Neural Network model. OpenCV provides powerful image processing functions required for ORB Copy-Move Detection, while Pillow (PIL) performs image preprocessing and Error Level Analysis. NumPy

supports numerical computations required during image processing and deep learning operations. These software components together satisfy the computational requirements of the project.

The requirement analysis phase also identifies the hardware resources required for efficient execution of the application. Since the project performs image processing and deep learning computations, a computer with adequate RAM, processor speed, and storage capacity is necessary. However, because the CNN architecture used in the project is lightweight, the application can execute efficiently even on ordinary personal computers without requiring expensive GPU hardware. This improves the accessibility of the system and enables students and researchers to use the application with minimal hardware investment.

The project also considers the quality of the input data during requirement analysis. The uploaded image must be in a supported format such as JPG, JPEG, or PNG. Images with extremely low resolution or excessive compression may affect the prediction accuracy because important visual features may be lost during preprocessing. Therefore, users are encouraged to upload clear and high-quality images for better detection performance. During preprocessing, the uploaded image is standardized by resizing and converting it into a format suitable for further analysis. This preprocessing stage ensures consistent input for all image analysis modules.

Requirement analysis also includes identifying the processing sequence followed by the application. Once the image is uploaded, the application first extracts metadata information. It then performs Error Level Analysis to identify compression inconsistencies. The ORB algorithm subsequently detects duplicated feature regions that may indicate copy-move forgery. Finally, the processed image is supplied to the trained Convolutional Neural Network model, which generates the final prediction. This systematic processing pipeline ensures that every uploaded image undergoes comprehensive forensic analysis before classification. Security and reliability requirements are also considered during this phase. Since users upload image files to the application, the system must validate file formats before processing to prevent unsupported or invalid files from causing execution errors. Proper exception handling mechanisms should be implemented to ensure that the application continues functioning even when invalid images are uploaded. Similarly, temporary processing files generated during Error Level Analysis should be properly managed to avoid unnecessary storage consumption. These requirements improve the overall robustness and reliability of the application. Scalability is another important requirement identified during project planning. The modular architecture of the Fake Image Detection System allows additional forensic techniques and advanced deep learning models to be integrated without significantly modifying the existing application. Future versions may include Deepfake detection, video forgery detection, transformer-based image classification models, cloud deployment, and mobile application support. The modular design adopted during development therefore provides flexibility for future enhancements.

Performance requirements were also analysed before implementation. The system should generate prediction results within a few seconds after the user uploads an image. Efficient execution of Metadata Analysis, Error Level Analysis, ORB feature matching, and CNN prediction is necessary to ensure smooth user experience.

The lightweight CNN architecture and optimized image preprocessing techniques contribute to faster execution while maintaining satisfactory prediction accuracy.

Requirement analysis also emphasizes maintainability and portability. The project is developed using open-source software libraries, making it easy to update and maintain. Since Python is platform-independent, the application can be executed on Windows, Linux, and macOS operating systems with minimal modifications. This portability increases the usability of the system across different computing environments.

Finally, the requirement analysis phase confirms that the proposed Fake Image Detection System satisfies the objectives of developing an intelligent, reliable, accurate, and user-friendly image authentication application. By carefully identifying user requirements, software requirements, hardware requirements, processing requirements, and quality requirements before implementation, the project establishes a strong foundation for successful software development. The detailed analysis performed during this phase contributes significantly to the overall effectiveness, performance, and maintainability of the developed system.

2.1. Feasibility Study:

A feasibility study is an important activity carried out before the development of a software project. It is a systematic evaluation process used to determine whether the proposed system can be successfully developed, implemented, and maintained using the available resources. The primary objective of conducting a feasibility study is to identify potential challenges, estimate the required resources, analyse the expected benefits, and ensure that the project is practical from technical, economic, and operational perspectives. It helps organizations avoid unnecessary expenditure on projects that are not achievable or beneficial.

For the Fake Image Detection System, the feasibility study was conducted to evaluate whether the proposed solution could be implemented using available software tools, hardware resources, development time, and technical expertise. Since the project integrates Artificial Intelligence, Image Processing, and Digital Image Forensics, it was necessary to analyse the availability of suitable technologies, programming libraries, computing resources, and trained models before starting development.

The feasibility study also examined the expected benefits of the system in comparison with its development cost. The proposed application is intended to provide an efficient solution for detecting manipulated images using multiple forensic techniques while remaining simple enough for ordinary users. The results of the feasibility study indicated that the project is technically feasible, economically affordable, operationally practical, and capable of meeting its intended objectives. Therefore, the development of the Fake Image Detection System was considered both achievable and beneficial.

2.2.1. Technical Feasibility:

Technical feasibility determines whether the proposed project can be successfully developed using the available technology, software tools, hardware resources, and technical knowledge. It evaluates whether the development team possesses the required skills and whether the selected technologies are capable of supporting the project requirements.

The Fake Image Detection System is technically feasible because it is developed using well-established and widely adopted open-source technologies. The primary programming language used is Python, which provides excellent support for Artificial Intelligence, Machine Learning, Deep Learning, and Image Processing applications. Python offers numerous libraries that simplify the implementation of complex image analysis algorithms without requiring extensive low-level programming.

The project utilizes the Stream lit framework to create an interactive web-based graphical user interface. Stream lit enables rapid application development and allows users to upload images and receive analysis results through a browser. Since Stream lit is lightweight and easy to configure, it significantly reduces the complexity of frontend development.

The Convolutional Neural Network used in the project is implemented using TensorFlow and Kera's, which are among the most popular deep learning frameworks available today. These frameworks provide pre-built functions for model training, prediction, optimization, and performance evaluation. Similarly, OpenCV provides efficient image processing algorithms required for ORB feature extraction and copy-move detection, while Pillow (PIL) supports image preprocessing and Error Level Analysis. NumPy performs numerical computations necessary during image processing operations.

From the hardware perspective, the project does not require expensive computing infrastructure. The trained CNN model is lightweight and can execute efficiently on a standard personal computer equipped with moderate RAM and processing power. Therefore, specialized GPU hardware is not mandatory for normal application execution. This makes the project technically feasible for educational institutions and individual developers.

The modular architecture adopted during development further improves technical feasibility. Each module performs an independent function and can be modified or upgraded without affecting the remaining components. This modular approach simplifies debugging, testing, maintenance, and future enhancements. Therefore, from both software and hardware perspectives, the proposed Fake Image Detection System is technically feasible.

2.1.2. Economic Feasibility:

Economic feasibility evaluates whether the financial investment required for developing and maintaining the software is justified by the expected benefits. It analyses development costs, software licensing expenses, hardware requirements, maintenance costs, and operational expenditure.

The Fake Image Detection System is economically feasible because almost all software technologies used during development are open-source and freely available. Python, Stream lit, TensorFlow, Kera's, OpenCV, Pillow, and NumPy can be downloaded and used without purchasing commercial licenses. As a result, software acquisition costs are practically eliminated.

The project also does not require expensive hardware infrastructure. A standard desktop or laptop computer with moderate specifications is sufficient for both development and execution. Since no specialized servers

or dedicated GPU workstations are mandatory, the initial investment remains low. This significantly reduces the overall development cost of the project.

Maintenance costs are also minimal because the application uses open-source libraries that receive regular updates from active developer communities. Future software upgrades can be implemented without substantial financial expenditure. Since the application operates locally using Stream lit, there are no recurring cloud hosting charges unless cloud deployment is chosen in future versions.

The benefits provided by the system greatly outweigh its development cost. The application assists journalists, cybersecurity professionals, researchers, students, and investigators in verifying image authenticity quickly and efficiently. It reduces manual effort, improves detection accuracy, saves investigation time, and enhances digital trust. Therefore, the project provides high value while requiring relatively low financial investment, making it economically feasible.

2.1.2.1. Cost Benefit Analysis:

Cost Benefit Analysis is performed to compare the overall cost of developing the proposed system with the expected benefits obtained after implementation. This analysis helps determine whether the investment made during software development is justified by the long-term advantages.

The development cost of the Fake Image Detection System mainly consists of the developer's effort, personal computer usage, electricity consumption, internet connectivity, and project documentation. Since all software tools used are open-source, software licensing costs are eliminated. The project therefore requires only minimal financial investment.

The expected benefits of the system are substantial. The application automates the image verification process, reducing dependence on manual forensic analysis. It provides faster prediction results, improves accuracy through multiple forensic techniques, and supports users in identifying manipulated digital images. The system also serves as an educational platform for learning Artificial Intelligence, Deep Learning, Image Processing, and Digital Image Forensics.

Additional benefits include improved digital security, reduced spread of fake images, assistance during cybercrime investigations, support for journalism, and promotion of trustworthy digital communication. The modular architecture also allows future upgrades without major redevelopment costs.

Since the long-term benefits significantly exceed the relatively low development cost, the overall Cost Benefit Analysis clearly supports the implementation of the proposed Fake Image Detection System.

2.1.3. Operational Feasibility:

Operational feasibility evaluates whether the proposed system can be effectively used by its intended users in a real-world environment. It focuses on user acceptance, ease of operation, reliability, maintainability, and day-to-day usability.

The Fake Image Detection System has been designed with a simple and interactive Stream lit interface that allows users to upload images with minimal effort. No specialized training is required to operate the

application, making it suitable for both technical and non-technical users. The entire image verification process is automated, reducing manual intervention and improving operational efficiency.

The application provides clear prediction results along with confidence scores and technical analysis, enabling users to understand the basis of the system's decision. Error handling mechanisms prevent application failures when unsupported image formats are uploaded, improving reliability during regular operation.

The modular software architecture simplifies maintenance and future updates. Individual modules can be modified independently without affecting the overall system. This reduces maintenance complexity and improves long-term operational stability.

The proposed system can be deployed in educational institutions, research laboratories, digital forensic organizations, cybersecurity departments, media organizations, and government agencies. Its ability to perform rapid image authentication while maintaining ease of use makes it operationally practical for real-world applications.

In conclusion, the Fake Image Detection System satisfies all major operational requirements by providing an efficient, user-friendly, reliable, and maintainable platform for digital image authentication. Therefore, the project is considered operationally feasible for practical implementation.

2.2. Software Requirement Specification (SRS):

Software Requirement Specification (SRS) is a formal document that describes the complete requirements of a software system before its development. It acts as a blueprint for the software project by defining the functional behaviour, non-functional characteristics, hardware requirements, software requirements, user requirements, and system constraints. An SRS document serves as a communication medium between developers, testers, project managers, and end users, ensuring that everyone clearly understands the objectives and functionality of the proposed system.

For the Fake Image Detection System, the Software Requirement Specification defines all the technical and functional requirements necessary for developing an intelligent image authentication application. Since the project integrates Artificial Intelligence, Deep Learning, Image Processing, and Digital Image Forensics, it is important to document every system requirement before implementation. A well-prepared SRS helps developers design the application efficiently, simplifies testing, reduces development errors, and improves software quality.

The proposed system is designed to automatically analyse uploaded digital images using multiple forensic techniques. The software must support image uploading, metadata extraction, Error Level Analysis (ELA), ORB Copy-Move Detection, CNN-based image classification, and result visualization. The application should also provide a user-friendly graphical interface through Streamlit so that both technical and non-technical users can operate the system without difficulty.

The Software Requirement Specification also identifies the quality attributes expected from the application. The system should provide accurate predictions, execute image analysis within a reasonable time, handle invalid inputs properly, and maintain consistent performance during repeated execution. Furthermore, the application should be portable across different operating systems and should allow future integration of advanced AI algorithms and forensic techniques.

By defining these requirements before development, the project ensures that the final software product satisfies user expectations while remaining reliable, maintainable, efficient, and scalable. Therefore, the Software Requirement Specification forms one of the most important documents in the development of the Fake Image Detection System.

2.2.1. Functional Requirements:

Functional requirements describe the operations and services that the software must perform to satisfy user needs. They specify the behaviour of the application, the sequence of operations, user interactions, and system responses. Functional requirements define what the software should do rather than how it should do it.

In the Fake Image Detection System, functional requirements include image uploading, preprocessing, metadata extraction, Error Level Analysis, ORB feature analysis, CNN prediction, and result presentation. Each function contributes toward the accurate detection of manipulated images and ensures smooth interaction between the user and the application.

2.2.1.1. Input:

Input requirements describe the information supplied to the application for processing. The quality and format of the input directly influence the performance and prediction accuracy of the Fake Image Detection System.

The primary input to the application is a digital image uploaded by the user. The application supports commonly used image formats including JPG, JPEG, and PNG. After selecting an image through the Streamlit interface, the application validates the uploaded file before further processing. Unsupported formats are rejected with appropriate error messages to prevent execution failures.

The uploaded image should have adequate resolution and image quality to enable effective forensic analysis. Extremely blurred or heavily compressed images may reduce prediction accuracy because important visual features become difficult to analyse. Therefore, users are encouraged to upload clear images captured using digital cameras or smartphones.

Once the image is accepted, it becomes the primary input for all image analysis modules, including Metadata Analysis, Error Level Analysis, ORB Copy-Move Detection, and CNN classification.

2.2.1.2. Output:

The output requirements define the information generated by the application after successful execution. The Fake Image Detection System provides both visual and textual outputs to help users understand the authenticity of the uploaded image.

The primary output is the final prediction, where the uploaded image is classified as either Real or Fake. Along with the prediction, the application displays the confidence score generated by the CNN model, indicating the reliability of the prediction.

The system also provides additional forensic outputs, including:

- Uploaded image preview.
- Extracted metadata information.
- Error Level Analysis (ELA) image.

- ORB feature matching analysis.
- Technical explanation of the prediction.
- Overall forensic analysis summary.

These outputs help users understand how the prediction was generated instead of simply displaying a final classification result.

2.2.1.3. Computational Requirements:

The computational requirements describe the processing activities performed by the application during execution.

After image upload, the system performs several computational operations including image preprocessing, metadata extraction, JPEG recompression for Error Level Analysis, ORB feature extraction, descriptor matching, CNN preprocessing, feature extraction, and binary image classification.

The Convolutional Neural Network performs multiple mathematical operations such as convolution, activation, pooling, flattening, and dense layer computations before generating the final prediction. OpenCV executes feature extraction algorithms, while NumPy performs numerical computations required during image processing.

The computational complexity of the project remains moderate because the CNN model is lightweight and optimized for efficient execution. Consequently, the application provides prediction results within a few seconds on standard personal computers.

2.2.1.4. Storage (Database Requirements):

Unlike many conventional software applications, the Fake Image Detection System does not require a relational database such as MySQL, PostgreSQL, or Oracle for storing user information or transactional data. The application primarily processes images in real time without permanently storing uploaded files. The uploaded image is temporarily stored in memory during processing. Intermediate files such as the recompressed image used for Error Level Analysis are created temporarily and deleted after execution. The trained CNN model is stored as a model file and loaded whenever the application starts.

The project stores the following resources:

- Trained CNN model file.
- Python source code.
- Image processing modules.
- Temporary uploaded image.
- Temporary ELA image.
- Required Python libraries.
- Project dataset used during training.

Since permanent database storage is not required, the project remains lightweight, consumes less storage space, and simplifies software deployment. Future versions may integrate cloud databases to maintain prediction history, user authentication, and image logs if required.

2.2.2. Non-Functional Requirements:

Non-functional requirements specify the quality attributes and performance characteristics of a software system. Unlike functional requirements, which define what the system should do, non-functional requirements describe how efficiently, reliably, securely, and effectively the system should perform its functions. These requirements play a crucial role in determining the overall quality, usability, and maintainability of the application.

For the Fake Image Detection System, non-functional requirements ensure that the application not only produces accurate predictions but also delivers a satisfactory user experience. The system should respond quickly to user requests, operate reliably under normal conditions, maintain consistent performance, and be easy to use and maintain. Since the application is intended for users with different technical backgrounds, simplicity and accessibility are equally important considerations.

The major non-functional requirements identified for the Fake Image Detection System include quality metrics, reliability, usability, response time, throughput, availability, and maintainability.

2.2.2.1. Quality Metrics:

Quality metrics are measurable standards used to evaluate the overall performance and quality of a software application. These metrics help developers determine whether the software satisfies user expectations and project objectives.

The Fake Image Detection System should provide accurate predictions while maintaining consistent performance throughout its execution. The software should produce reliable outputs for valid input images and minimize incorrect classifications. The graphical user interface should remain responsive during image processing, and the application should display results in a clear and understandable manner.

Important quality metrics considered for this project include prediction accuracy, execution speed, software reliability, user friendliness, code maintainability, and system stability. Regular testing and validation ensure that these quality standards are maintained throughout software development and deployment.

2.2.2.2. Reliability:

Reliability refers to the ability of the software system to perform its intended functions consistently without failures under specified operating conditions. A reliable application generates accurate results repeatedly and handles unexpected situations without crashing.

The Fake Image Detection System has been designed to provide consistent image classification results whenever valid images are uploaded. Exception handling mechanisms prevent application failures caused by unsupported image formats or corrupted files. The modular architecture also improves reliability by isolating different processing stages, allowing errors in one module to be identified and corrected without affecting the remaining components.

The use of proven libraries such as TensorFlow, OpenCV, Pillow, and NumPy further enhances software reliability because these libraries have been extensively tested by the developer community.

2.2.2.3. Usability:

Usability refers to the ease with which users can understand, learn, and operate the software application. A highly usable system requires minimal training and enables users to accomplish their tasks efficiently.

The Fake Image Detection System emphasizes simplicity through its Streamlit-based graphical user interface. Users only need to upload an image using the provided file uploader, after which the application automatically performs all forensic analysis steps. The prediction results, confidence score, metadata information, ELA output, and ORB analysis are displayed in a structured format that is easy to understand. The application is suitable for students, researchers, journalists, cybersecurity professionals, and general users because it does not require prior knowledge of Artificial Intelligence or Digital Image Forensics.

2.2.2.4. Response Time:

Response time is the duration required by the application to process user requests and generate results. Faster response times improve user satisfaction and increase software efficiency.

The Fake Image Detection System is designed to provide prediction results within a few seconds after image upload. The lightweight CNN model, efficient preprocessing algorithms, and optimized Python libraries reduce computation time while maintaining prediction accuracy.

Response time depends on factors such as image resolution, processor speed, and available system memory. However, under normal operating conditions, the application generates results quickly enough for practical real-world usage.

2.2.2.5. Throughput:

Throughput represents the amount of work performed by the software within a given period of time. It indicates the number of tasks or images that can be processed efficiently.

The Fake Image Detection System processes uploaded images sequentially. Since image preprocessing and CNN prediction require relatively low computational resources, multiple images can be analysed one after another without significant performance degradation. The lightweight architecture enables efficient utilization of processor and memory resources, resulting in satisfactory throughput for educational, research, and small-scale forensic applications.

Future versions may increase throughput by implementing parallel processing techniques or deploying the application on cloud computing platforms.

2.2.2.6. Availability:

Availability refers to the readiness of the software system to provide its services whenever users require them. High availability ensures that the application remains operational with minimal downtime.

The Fake Image Detection System is designed to operate continuously on supported operating systems. Since it is implemented using Stream lit, the application can be executed locally whenever the required

software dependencies are installed. The application does not depend on continuous internet connectivity during local execution, improving its availability.

Future cloud deployment can further improve availability by enabling users to access the application from different locations through web browsers.

2.2.2.7. Maintainability:

Maintainability refers to the ease with which software can be modified, corrected, updated, or enhanced after deployment. Well-structured software reduces maintenance effort and simplifies future development.

The Fake Image Detection System follows a modular programming approach in which individual components such as Metadata Analysis, Error Level Analysis, ORB Detection, and CNN Prediction are implemented separately. This architecture allows developers to update or improve one module without affecting the remaining components.

Python's simple syntax, extensive documentation, and large developer community further simplify software maintenance. Additional forensic techniques, improved CNN models, and cloud integration can be incorporated into future versions without major modifications to the existing codebase.

2.3.3 Environment & Technology Requirements:

The development of the Fake Image Detection System requires a suitable software environment and modern technologies capable of supporting Artificial Intelligence, Image Processing, and Web Application Development.

The project is developed using the Python programming language, which provides powerful libraries for machine learning and image analysis. The graphical user interface is developed using Stream lit, enabling users to interact with the application through a web browser.

The CNN model is implemented using TensorFlow and Keras, while OpenCV performs image processing and ORB feature extraction. Pillow (PIL) is used for image manipulation and Error Level Analysis, and NumPy supports numerical computations.

The development environment includes Visual Studio Code as the code editor, Python Virtual Environment for dependency management, and GitHub for project version control. These technologies collectively provide a stable and efficient environment for software development.

2.2.3.1. Hardware Requirements:

The Fake Image Detection System requires only moderate hardware resources for development and execution.

The minimum hardware requirements are:

Processor: Intel Core i3 (8th Generation) or AMD equivalent (Intel Core i5 or above recommended)

RAM: Minimum 4 GB (8 GB recommended)

Storage: At least 20 GB free disk space

Display: 1366×768 resolution or higher

Internet Connection: Required for installing Python libraries and project dependencies

Input Device: Keyboard and Mouse

Output Device: Monitor

These hardware specifications are sufficient for executing the Stream lit application and running the trained CNN model efficiently.

2.2.3.2. Software Requirements:

The software requirements specify all operating systems, programming languages, frameworks, and libraries required for successful project execution.

The Fake Image Detection System requires the following software:

Operating System: Windows 10/11, Ubuntu Linux, or macOS

Programming Language: Python 3.10 or above

IDE/Editor: Visual Studio Code

Framework: Stream lit

Deep Learning Framework: TensorFlow and Kera's

Image Processing Library: OpenCV

Image Manipulation Library: Pillow (PIL)

Numerical Computing Library: NumPy

Package Manager: pip

Version Control: Git and GitHub

Web Browser: Google Chrome, Microsoft Edge, or Mozilla Firefox

These software components work together to provide a complete environment for developing, testing, and executing the Fake Image Detection System.

3.DESIGN

3.1 System Design:

System design is one of the most important phases of the Software Development Life Cycle (SDLC). After gathering and analysing the system requirements, the next step is to transform those requirements into a structured design that serves as a blueprint for software development. System design defines the overall architecture, components, modules, data flow, user interaction, and implementation strategy of the software system. It acts as a bridge between requirement analysis and coding by providing developers with a clear understanding of how the application should be built.

The primary objective of system design is to develop an efficient, reliable, maintainable, and scalable software architecture capable of satisfying all user requirements. A well-designed system reduces development complexity, simplifies software testing, improves maintainability, and minimizes implementation errors. It also provides proper documentation that helps future developers understand and enhance the application.

The Fake Image Detection System follows a modular design approach. Instead of implementing the entire application as a single program, the system is divided into several independent modules, each responsible for performing a specific task. This modular architecture improves software organization, allows independent testing of each module, and simplifies future upgrades.

The workflow of the proposed system begins when the user accesses the application through the Streamlit web interface. The user uploads a digital image in one of the supported formats such as JPG, JPEG, or PNG. The uploaded image first undergoes validation to ensure that the file format is supported and that the image can be processed correctly. Invalid or corrupted images are rejected with an appropriate error message to prevent system failure.

After successful validation, the uploaded image is forwarded to the preprocessing stage. During preprocessing, the image is converted into the required format, resized whenever necessary, and prepared for further forensic analysis. This stage ensures that all uploaded images maintain consistent dimensions and quality before entering the Artificial Intelligence model.

The first analysis module performs Metadata Analysis. This module extracts EXIF metadata from the uploaded image using Python image processing libraries. Information such as camera model, image resolution, creation date, software details, and other available metadata are displayed to the user. Missing or suspicious metadata may provide useful indications regarding possible image manipulation.

The second module performs Error Level Analysis (ELA). The uploaded image is recompressed using JPEG compression and compared with the original image. Differences between the two images are amplified to produce the ELA image. Regions showing unusual compression differences may indicate image tampering or editing operations.

The third module performs ORB Copy-Move Detection. Using OpenCV, the ORB algorithm extracts feature points from the image and compares feature descriptors to identify duplicated image regions. If similar key

points appear at different locations within the image, the system considers them as potential indicators of copy-move forgery.

After completing forensic analysis, the processed image is supplied to the trained Convolutional Neural Network (CNN) model. The CNN automatically extracts visual features using multiple convolutional layers and predicts whether the uploaded image is Real or Fake. Along with the prediction, the model generates a confidence score representing the probability of the classification result.

The final stage displays all analysis results through the Streamlit interface. Users can view the uploaded image, extracted metadata, ELA image, ORB analysis, prediction result, confidence score, and technical explanation generated by the system. This integrated presentation allows users to understand the basis of the prediction rather than relying only on the final classification.

One of the major strengths of the proposed design is its modular architecture. Since each module operates independently, developers can modify or upgrade individual modules without affecting the remaining system. For example, the CNN model can be replaced with a more advanced deep learning architecture while retaining the Metadata Analysis and ORB Detection modules. Similarly, additional forensic techniques such as Deepfake detection, GAN artifact analysis, frequency-domain analysis, or cloud-based verification can be incorporated in future versions with minimal architectural modifications.

The proposed system design therefore provides a reliable, flexible, scalable, and maintainable framework for intelligent fake image detection. It effectively integrates Artificial Intelligence, Digital Image Processing, and Digital Image Forensics into a single software application capable of assisting users in verifying image authenticity accurately and efficiently.

3.1.1 Introduction to UML:

Unified Modeling Language (UML) is a standardized graphical modeling language used for analyzing, designing, visualizing, and documenting software systems. UML helps developers understand the structure and behavior of a system before implementation. It improves communication among developers, designers, testers, and users by providing a common visual representation of the software.

In the Fake Image Detection System, UML diagrams are used to represent different aspects of the application such as user interaction, system workflow, object relationships, and processing sequence. These diagrams simplify the understanding of the project architecture and make future maintenance easier.

3.1.2. UML Diagrams:

3.1.2.1. Scenario:

Scenario Name

Detection of Fake Images

Actor

User

Preconditions

Stream lit application is running.

CNN model (model.h5) is loaded.

User has a valid image.

Main Flow

User opens the application.

User uploads an image.

System checks image metadata.

System performs Error Level Analysis (ELA).

System performs ORB Copy-Move Detection.

CNN predicts the image class.

Final result is displayed.

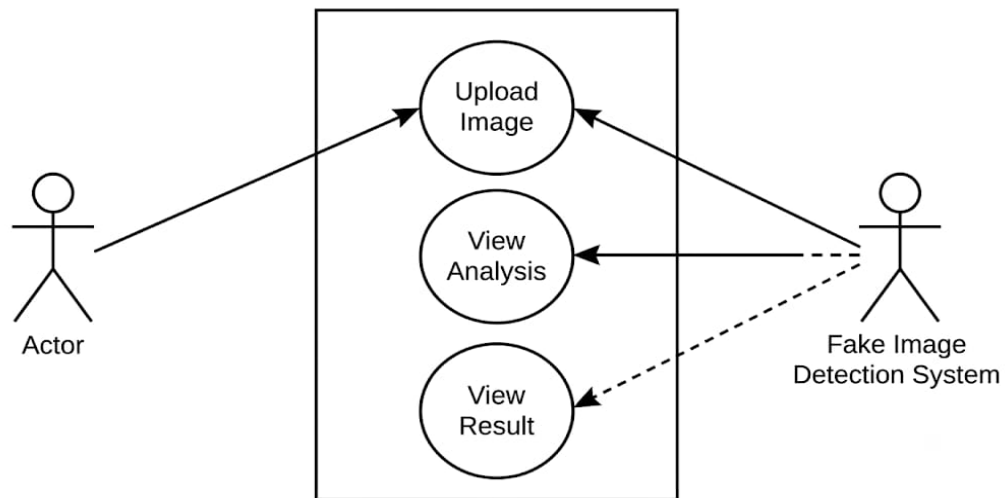
Post Conditions

Prediction is displayed.

Technical analysis is shown.

User can upload another image.

3.1.2.2. Use-Case Diagram:



Description:

The Use Case Diagram illustrates the interaction between the user and the Fake Image Detection System. The use case diagram provides a clear representation of the overall functionality of the Fake Image Detection System and the interaction between the user and the application. Each use case represents a specific function that contributes to the image verification process. The system integrates multiple forensic techniques, including metadata analysis, Error Level Analysis (ELA), Copy-Move Forgery Detection, and CNN-based classification, to improve detection accuracy. The generated technical analysis and final prediction help users understand whether the uploaded image is authentic or manipulated. This design ensures that the system is user-friendly, efficient, and capable of providing reliable image authenticity verification.

Use Cases:

Upload Image

View Uploaded Image

Perform Metadata Analysis

Perform ELA Analysis

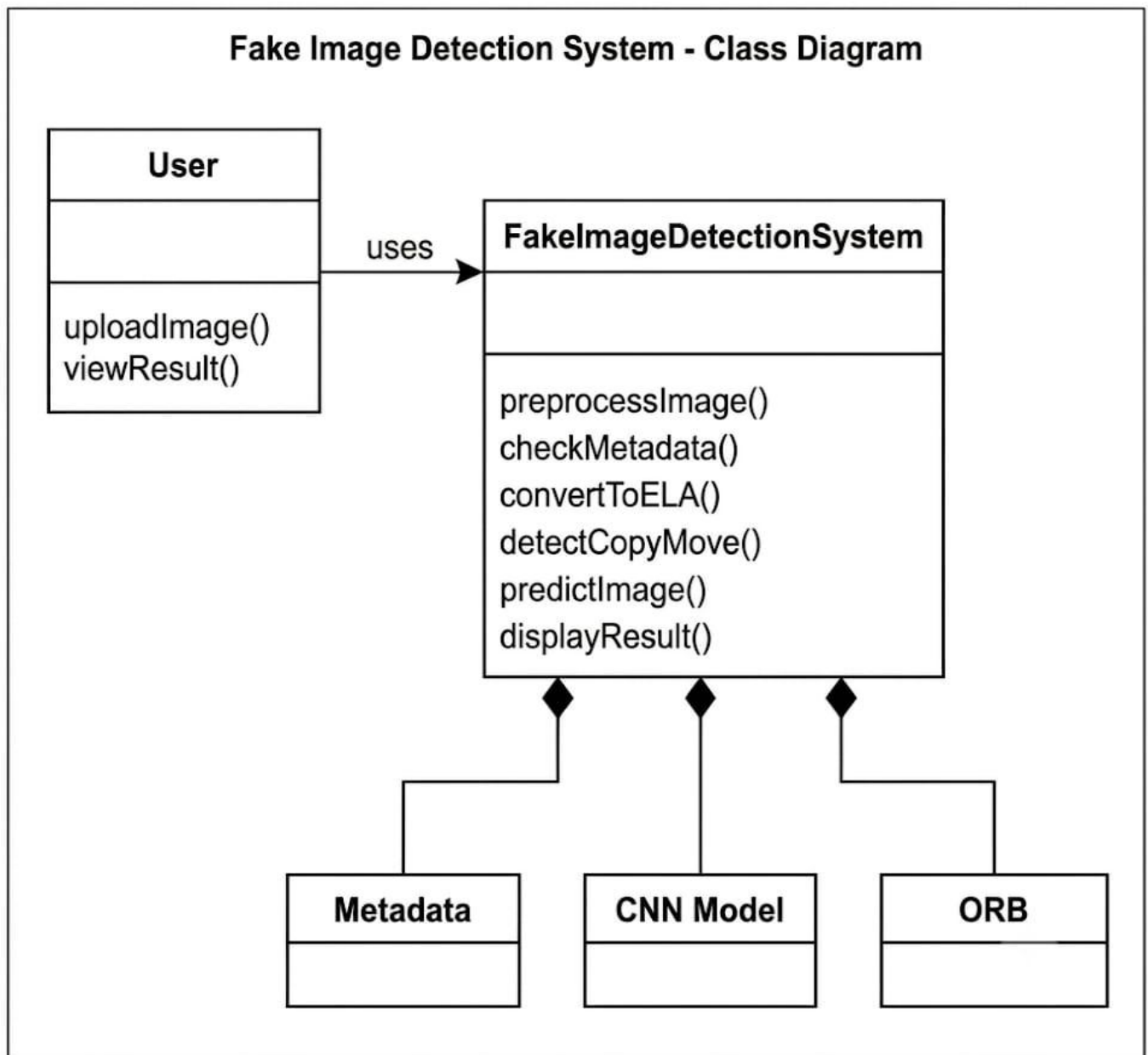
Detect Copy-Move Forgery

CNN Prediction

Display Technical Analysis

Display Final Result

3.1.2.3 Class Diagram:

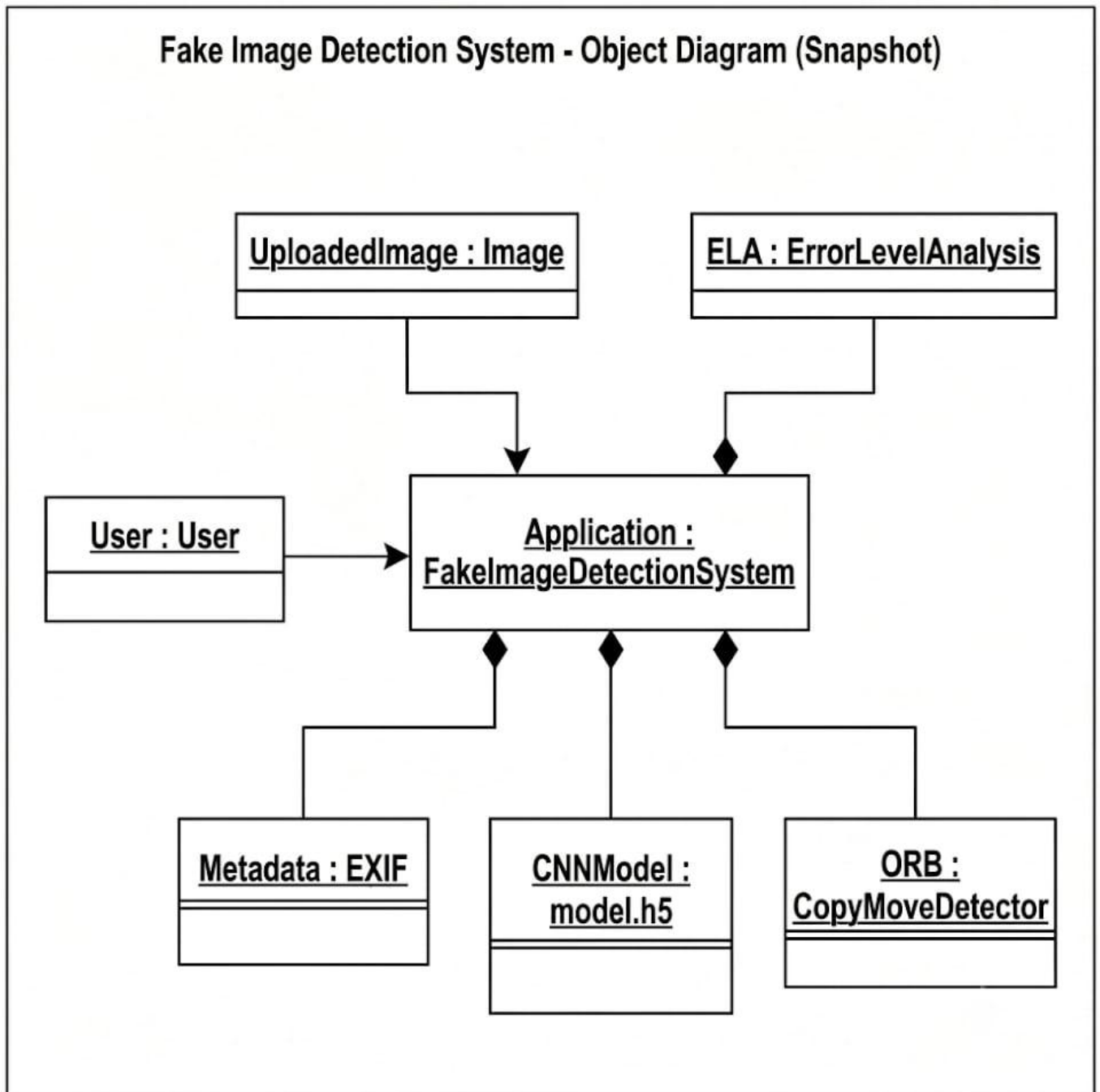


Description:

The User interacts with the Fake Image Detection System class. The system internally communicates with Metadata Analysis, CNN Model, and ORB Detection modules.

The class diagram also represents the relationships between the core system components and highlights the modular design of the application. Each module performs a specific task in the image analysis process, making the system easy to understand, maintain, and extend. This separation of responsibilities improves code reusability, enhances system performance, and allows future integration of additional image forensic techniques without affecting the existing functionality.

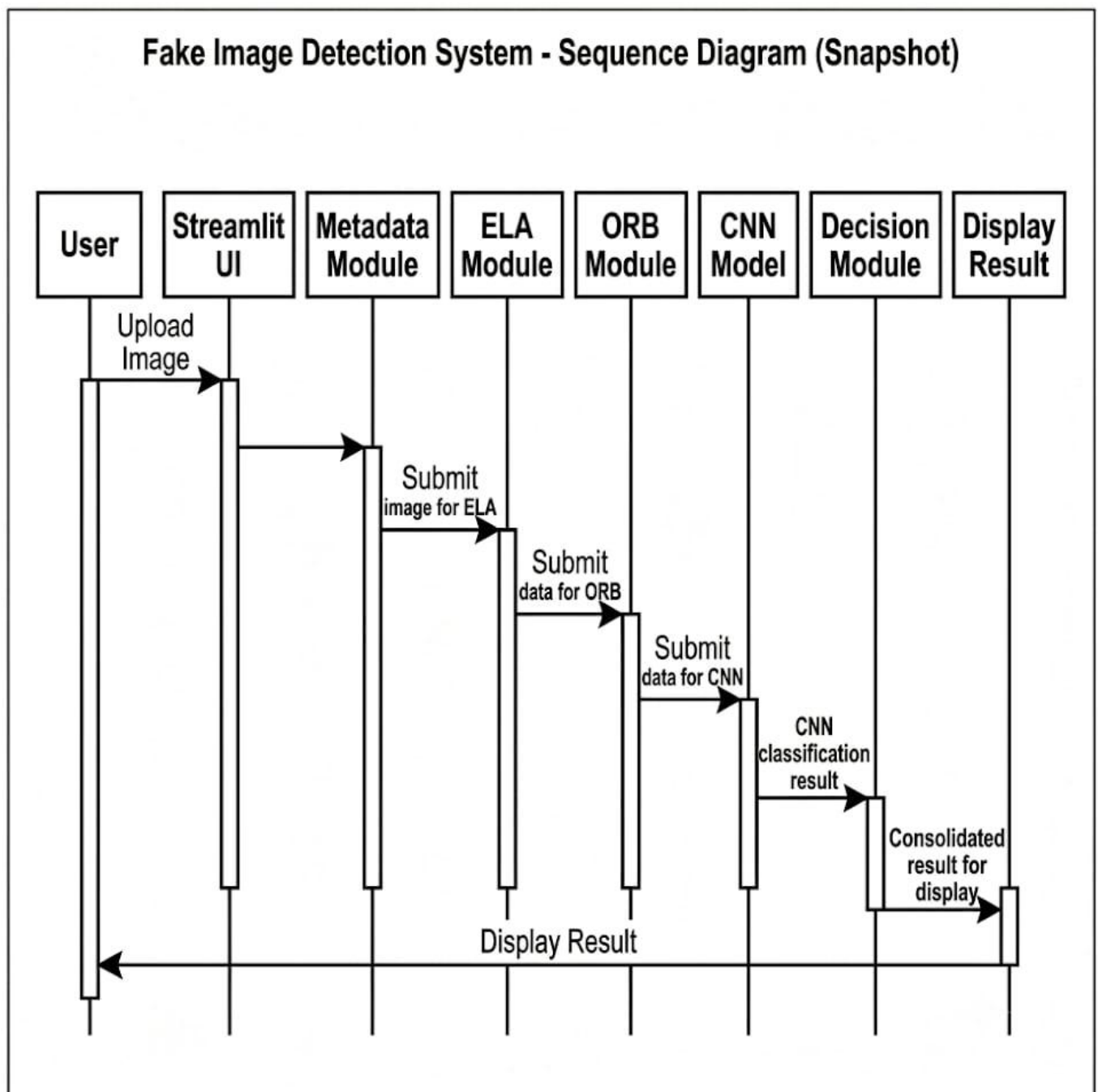
3.1.2.4. Object Diagram:



Description:

The Object Diagram illustrates the runtime objects involved in the Fake Image Detection System and their interactions during image processing.

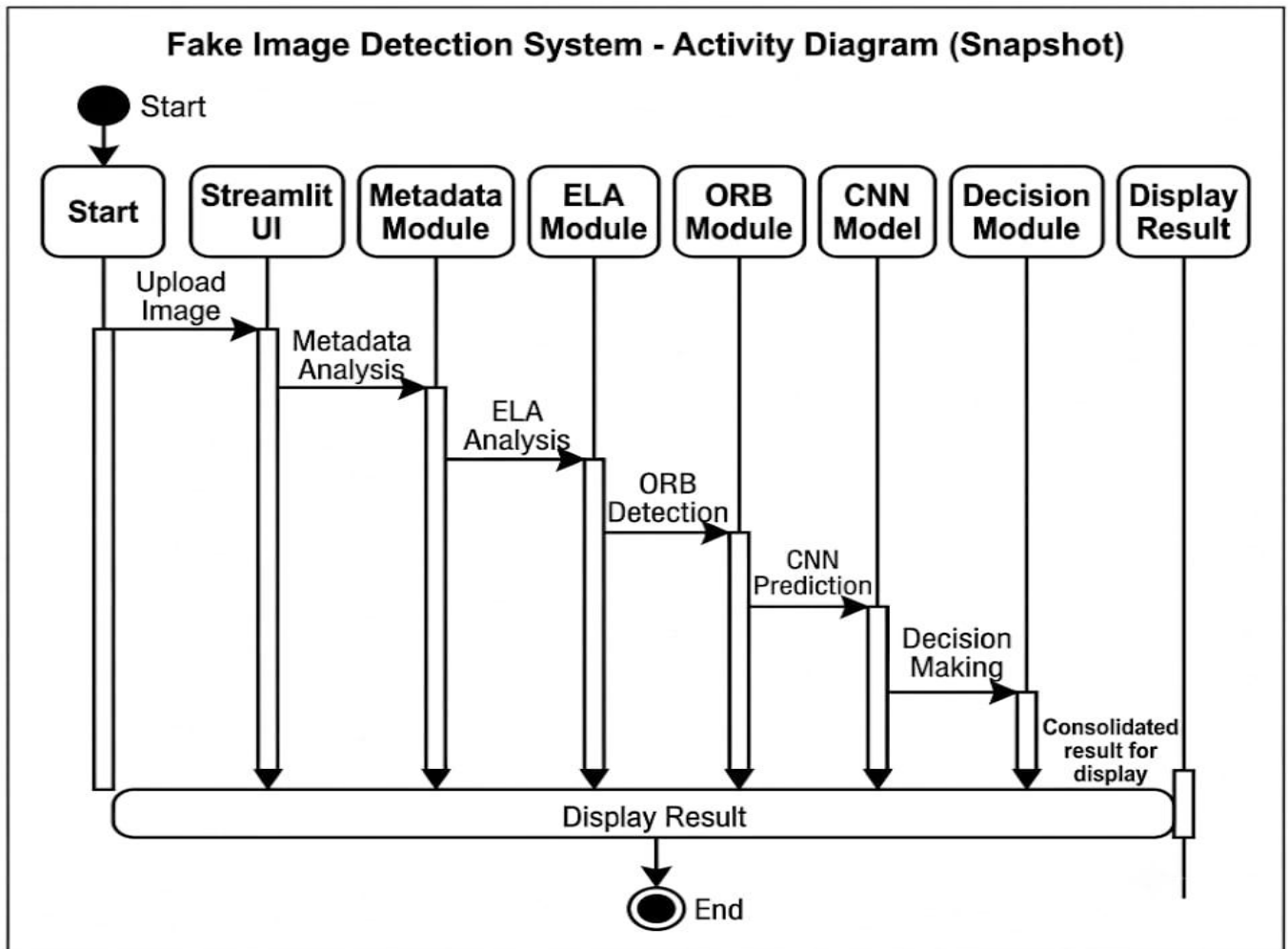
3.1.2.5. Sequence Diagram:



Description:

The sequence diagram illustrates the chronological flow of operations in the Fake Image Detection System. It shows how the user uploads an image through the Stream lit interface, after which the image is processed sequentially by the Metadata, ELA, ORB, and CNN modules. Finally, the Decision Module generates the prediction and displays the result to the user, representing the complete execution flow of the system.

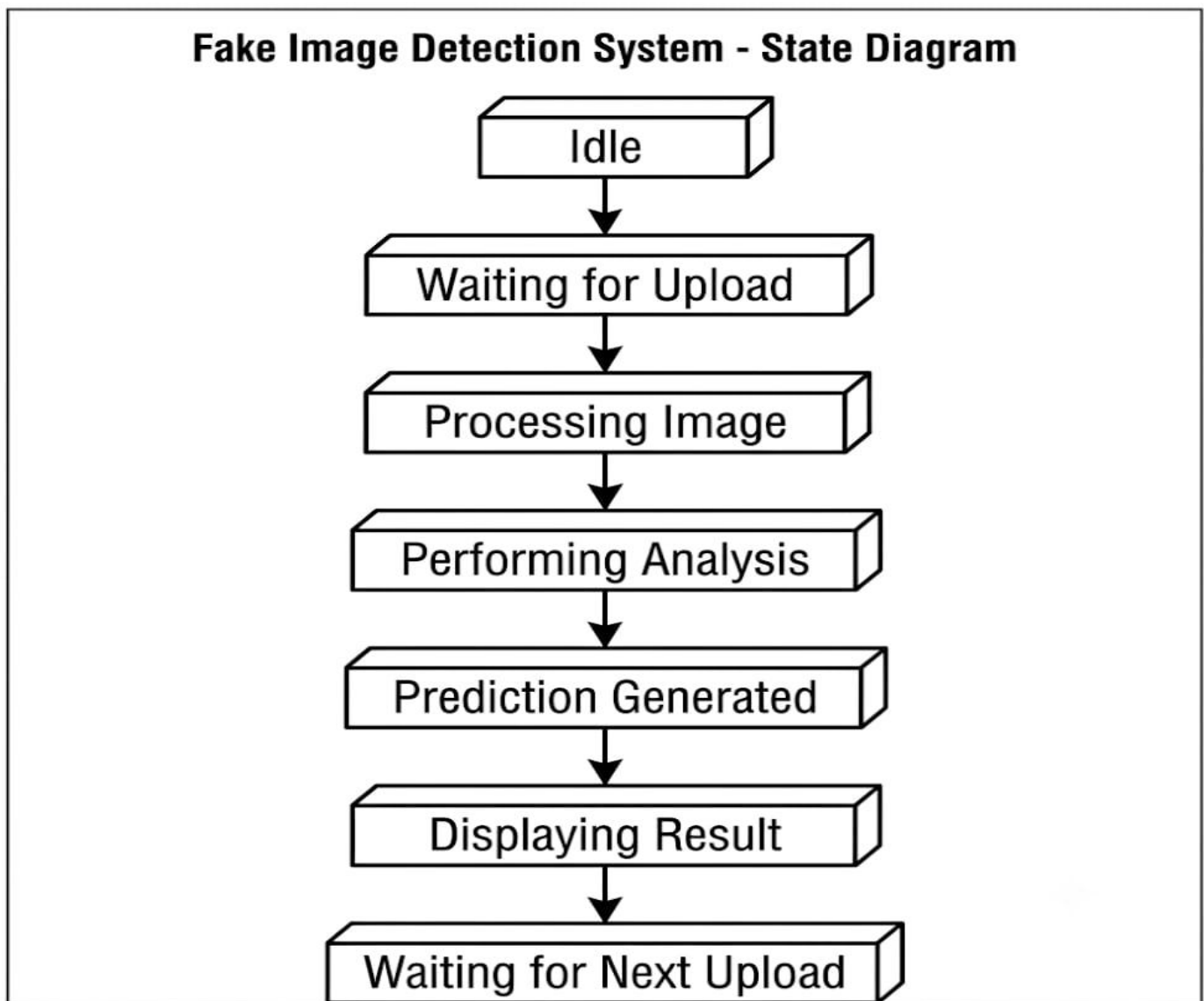
3.1.2.6. Activity Diagram:



Description:

The Activity Diagram illustrates the complete workflow of the Fake Image Detection System from the beginning of the image verification process to the final prediction. The process starts when the user uploads an image through the application interface. The uploaded image is first analyzed by the Metadata Analysis module, which extracts EXIF information and checks for any missing or suspicious metadata. The image is then processed using Error Level Analysis (ELA) to identify compression inconsistencies that may indicate digital manipulation. Next, the ORB Detection module examines the image to detect copy-move forgery by identifying duplicated regions. After the forensic analyses are completed, the processed image is passed to the CNN Prediction module, which classifies the image as either real or fake based on learned image features. The Decision-Making module combines the outputs of all analysis modules to generate the final prediction. Finally, the result is displayed to the user, and the activity reaches the End state. The activity diagram clearly represents the sequential flow of operations and helps in understanding the overall execution process of the application.

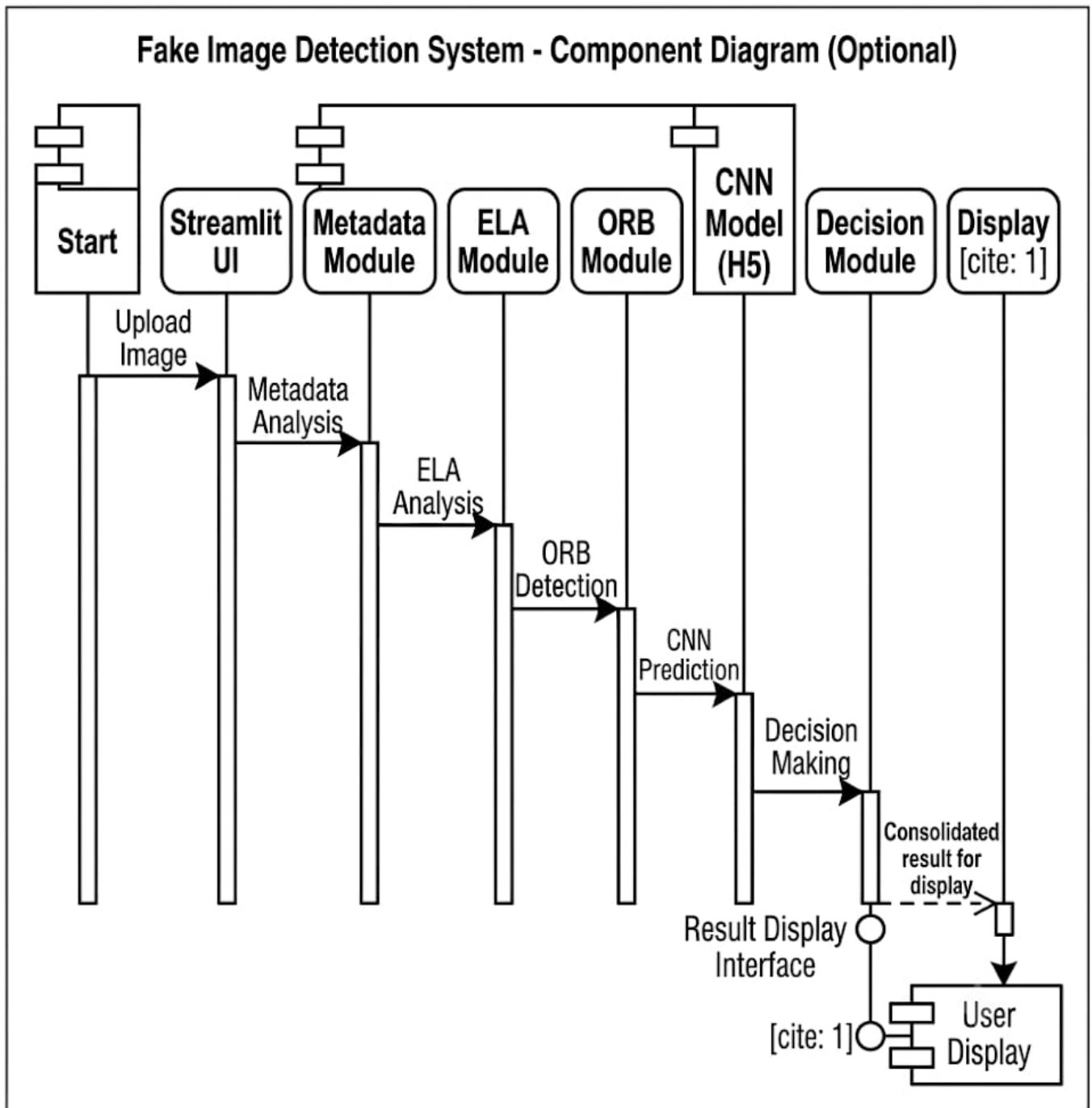
3.1.2.7. State diagram:



Description:

The State Diagram illustrates the different states of the Fake Image Detection System during its execution. The system initially remains in the Idle state and transitions to Waiting for Upload until the user selects an image. After an image is uploaded, the system enters the Processing Image state, where preprocessing is performed before moving to Performing Analysis, which includes metadata verification, Error Level Analysis (ELA), ORB copy-move detection, and CNN-based classification. Once the analysis is complete, the system enters the Prediction Generated state and then proceeds to Displaying Result, where the final prediction and technical analysis are presented to the user. After displaying the result, the system returns to the Waiting for Next Upload state, making it ready to process another image. The state diagram clearly represents the transitions between different stages of the application's execution and demonstrates the continuous workflow of the system.

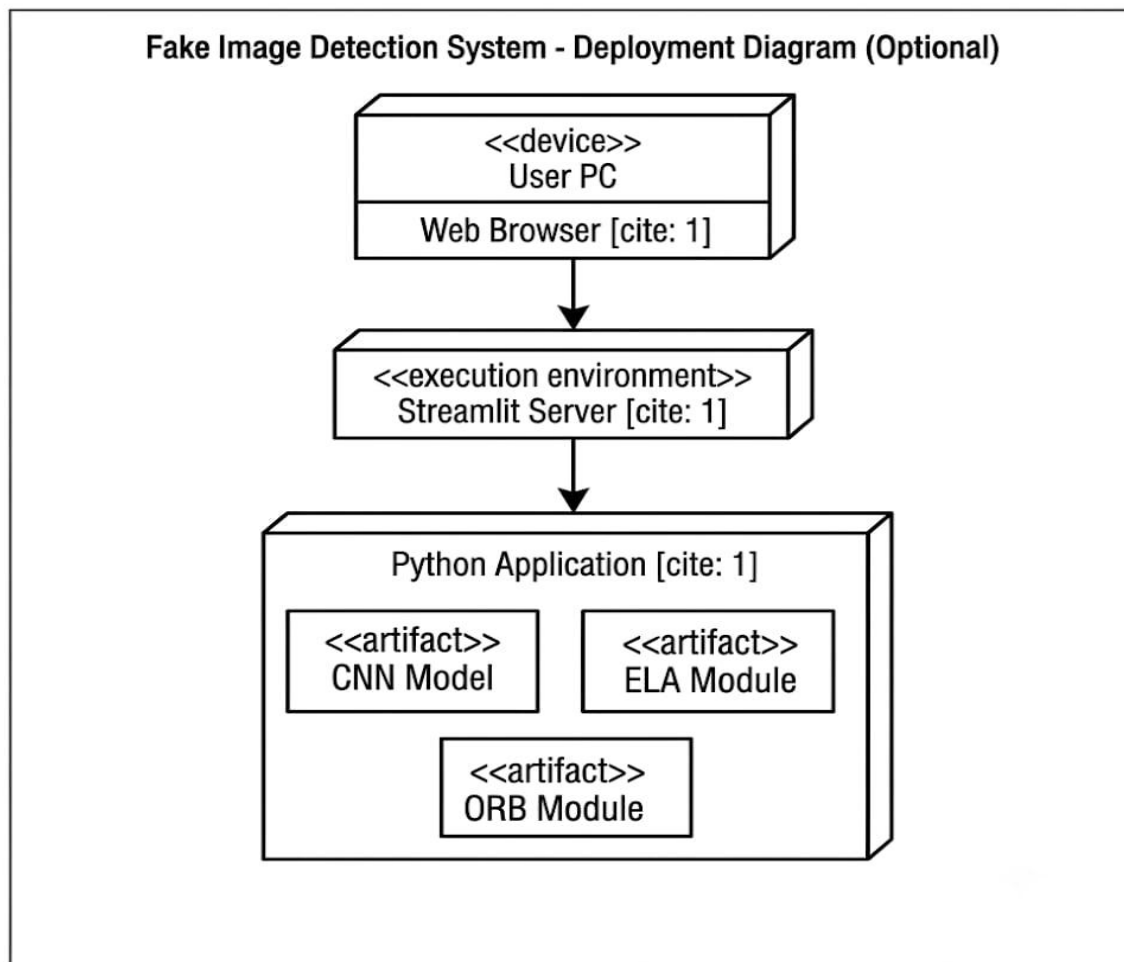
3.1.2.8. Component Diagram:



Description:

The Component Diagram illustrates the major software components of the Fake Image Detection System and their interactions. The Streamlit Interface serves as the user interface for uploading images and displaying results. The uploaded image is processed sequentially by the Metadata Module, ELA Module, ORB Module, and the CNN Model (H5) to perform forensic analysis and classification. Finally, the Result Module combines the outputs of all components and presents the prediction along with the analysis details to the user. The component diagram provides a high-level view of the system architecture and demonstrates how the individual modules work together to achieve accurate fake image detection.

3.1.2.9. Deployment Diagram:



Description:

The Component Diagram represents the high-level architecture of the Fake Image Detection System and illustrates how the major software components interact to perform image verification. The process begins with the Streamlit Interface, which provides a user-friendly platform for uploading images and viewing the analysis results. Once an image is uploaded, it is passed to the Metadata Module, where EXIF information is extracted and verified to detect any missing or suspicious metadata. The image is then forwarded to the ELA Module, which performs Error Level Analysis to identify compression inconsistencies that may indicate image manipulation. Next, the ORB Module detects copy-move forgery by identifying duplicated regions within the image. The processed image is subsequently provided to the CNN Model (H5), which classifies the image as Real or Fake based on features learned during training. Finally, the Result Module combines the outputs of all analysis components and presents the final prediction along with the technical analysis to the user. The component diagram clearly demonstrates the modular architecture of the system, where each component performs a specific task, resulting in a scalable, maintainable, and efficient fake image detection application.

3.2 Interface Design (Input Forms):

Interface Design is an important phase in software development that focuses on creating an effective communication medium between the user and the software application. A well-designed interface improves usability, reduces user errors, and enhances the overall user experience. The primary objective of interface design is to ensure that users can interact with the system easily without requiring technical knowledge. The Fake Image Detection System has been developed using the Stream lit framework, which provides a simple, responsive, and interactive web interface. The interface allows users to upload an image, perform image analysis, and view the prediction results through an organized layout. The design emphasizes simplicity, making the application suitable for students, researchers, journalists, cybersecurity professionals, and general users.

The interface follows a sequential workflow. The user first uploads an image using the file uploader. After the image is uploaded successfully, the application performs Metadata Analysis, Error Level Analysis (ELA), ORB Copy-Move Detection, and CNN Prediction automatically. Finally, all analysis results are displayed on the same page, allowing users to understand how the prediction was generated.

The interface has been designed to minimize user interaction. Instead of requiring multiple buttons or configuration settings, the application performs all image analysis operations automatically after the image is uploaded. This approach improves usability and reduces the possibility of user mistakes.

3.2.1 Home Page:

The Home Page serves as the main entry point of the application. It displays the project title, brief instructions, and the image upload option. Users can access all application features from this page.

Features:

Displays project title.

Shows project description.

Provides image upload option.

Simple and user-friendly layout.

Responsive Stream lit interface.

3.2.2 Image Upload Form:

The Image Upload Form allows users to select an image from their local computer. The application supports JPG, JPEG, and PNG image formats.

After selecting the image, the system validates the file before beginning the analysis process. Unsupported image formats are rejected with an appropriate error message.

Input Fields

Field

Description

Upload Image

Select JPG, JPEG or PNG image

Supported Formats

JPG, JPEG, PNG

Figure 3.1 Image Upload Interface



Upload an Image

+-----+

| Choose File |

+-----+

Supported Formats:

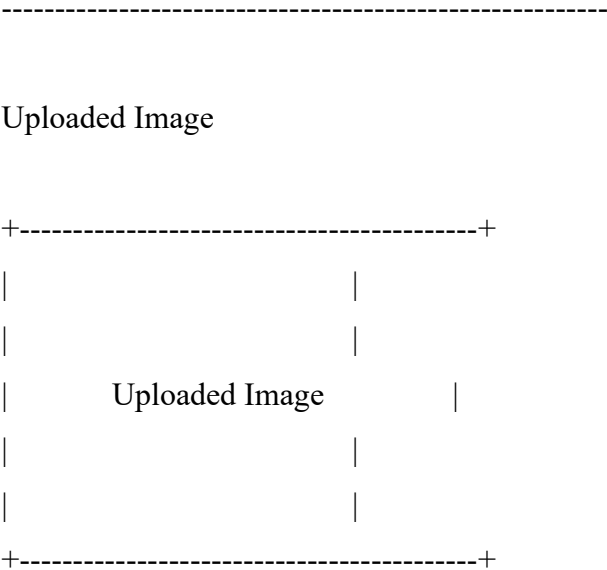
- ✓ JPG
- ✓ JPEG
- ✓ PNG

3.2.3 Uploaded Image Preview:

After uploading, the application displays the selected image before performing analysis. This helps users verify that the correct image has been uploaded.

The preview module improves usability by allowing visual confirmation before prediction.

Figure 3.2 Uploaded Image Preview



Status:

Image Uploaded Successfully

3.2.4 Analysis Interface:

Once the image is uploaded, the application automatically performs all forensic analyses.

The following analyses are executed:

Metadata Analysis

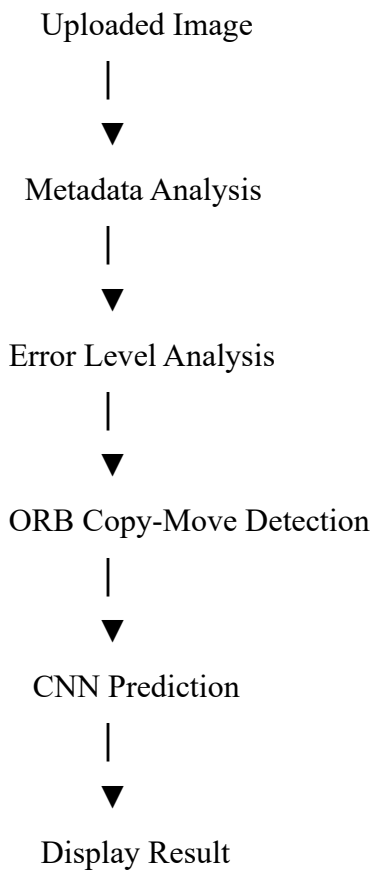
Error Level Analysis (ELA)

ORB Copy-Move Detection

CNN Prediction

Each analysis is performed sequentially and displayed on the interface.

Figure 3.3 Image Analysis Process



3.2.5 Result Display Interface:

After completing all analyses, the application displays the prediction results.

The result page includes:

Uploaded Image

Metadata Information

ELA Image

ORB Analysis

Prediction (Real/Fake)

Confidence Score

The user can understand the prediction through the displayed forensic evidence.

Figure 3.4 Result Screen

Prediction Result

Uploaded Image: ✓

Prediction:

REAL IMAGE

Confidence:

98.75 %

Metadata:

Available

ELA:

Generated Successfully

ORB:

No Suspicious Copy-Move Detected

Advantages of the Interface Design

Simple and interactive interface.

Easy image uploading process.

Automatic image analysis.

Clear display of prediction results.

No technical knowledge required.

Fast response and smooth navigation.

Supports multiple image formats.

Easy to maintain and upgrade.

3.3 Database Design:

Database Design is the process of organizing and managing data so that it can be stored, retrieved, and processed efficiently. In traditional software applications, a relational database such as MySQL, Oracle, PostgreSQL, or SQL Server is used to store user information, transactions, and application data. However, Artificial Intelligence and Machine Learning applications often process data directly from files and trained models instead of using relational databases.

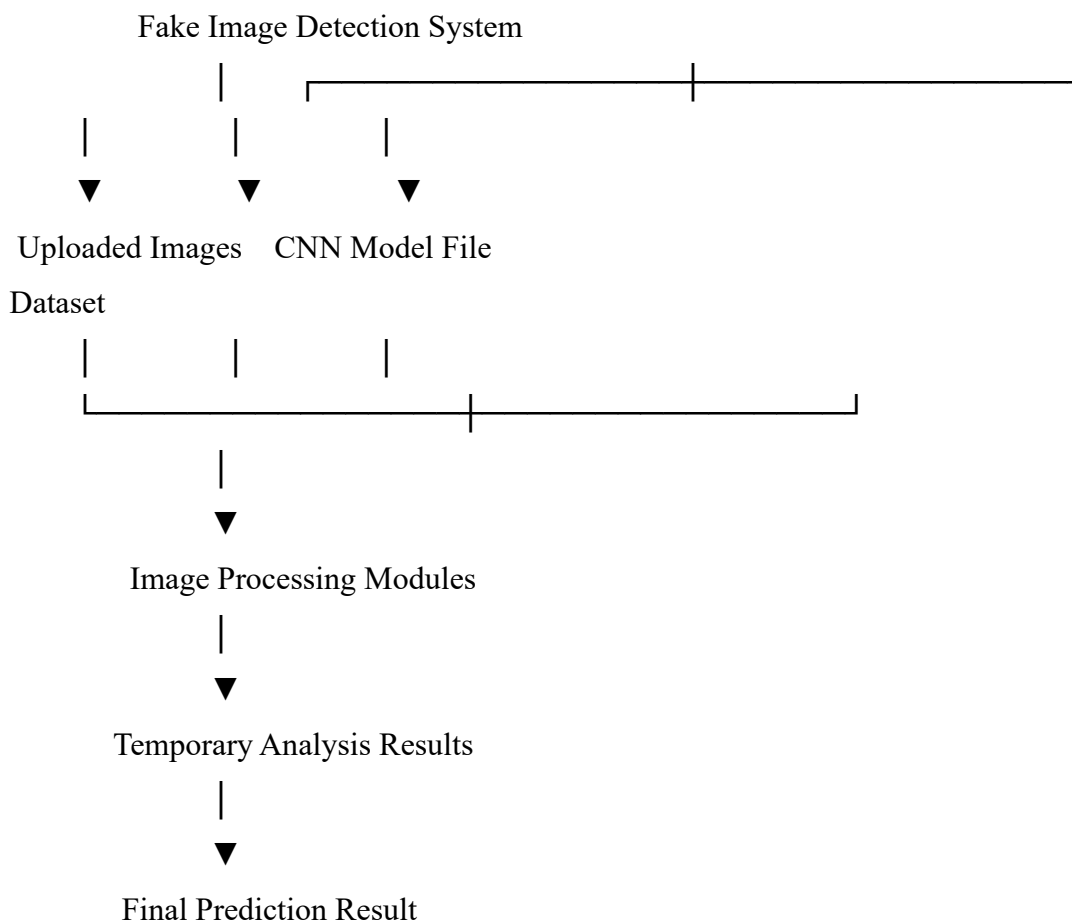
The Fake Image Detection System is primarily an image processing and deep learning application.

Therefore, it does not require a traditional database for its normal operation. Instead, the application stores information in the form of datasets, trained model files, temporary uploaded images, and generated analysis results. These files are processed during runtime and are not permanently stored unless required.

The project follows a file-based storage architecture, where uploaded images are processed temporarily, analyzed using different forensic techniques, and deleted after processing. The trained CNN model is stored as a separate model file and loaded automatically when the application starts. This approach reduces storage requirements, simplifies software deployment, and eliminates the need for database administration.

Although a relational database is not used, the system still maintains a logical organization of project files, datasets, and generated outputs. This structured organization ensures efficient execution, easier maintenance, and better scalability for future enhancements.

Figure 3.5 File Storage Architecture



3.3.1. Introduction to Backend:

The backend is the core processing component of the Fake Image Detection System. It performs all computational operations required for image authentication and manages communication between the user interface and Artificial Intelligence model. Unlike conventional web applications that rely on relational databases, the backend of this project focuses on image processing, machine learning prediction, and file management.

The backend has been developed using the Python programming language, which provides extensive support for Artificial Intelligence and Image Processing. Python enables the integration of multiple libraries such as TensorFlow, Keras, OpenCV, Pillow, NumPy, and Streamlit into a single application.

The backend executes the following sequence of operations:

Accept uploaded image.

Validate image format.

Extract image metadata.

Generate Error Level Analysis (ELA).

Perform ORB Copy-Move Detection.

Preprocess image for CNN.

Load trained CNN model.

Predict Real or Fake image.

Calculate confidence score.

Display final result.

The modular backend architecture improves software maintainability and allows future forensic techniques to be integrated without significant modifications.

3.3.2. Normalization Regarding the Project Tables:

Normalization is the process of organizing data to eliminate redundancy and improve data consistency. In traditional database systems, normalization involves dividing large tables into smaller related tables following normal forms such as First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF).

Since the Fake Image Detection System does not use a relational database, normalization is applied conceptually to the organization of project files and processing modules rather than SQL tables.

The project organizes its resources into separate logical components:

Dataset Folder

Trained CNN Model

Python Source Files

Uploaded Images

Temporary ELA Images

ORB Analysis Results

Output Prediction

This modular organization avoids duplication of data, improves code readability, simplifies maintenance, and supports future expansion of the application.

3.3.3. Table Description:

Although the project does not contain SQL database tables, the logical storage structure can be represented as follows.

Table 3.1 Image Information

Field Name
Data Type
Description
Image Name
String
Uploaded image file name
Image Format
String
JPG, JPEG or PNG
Image Size
Integer
Size of image file
Resolution
String
Width × Height
Upload Status
Boolean
Upload successful or not

Table 3.2 Metadata Information

Field Name
Data Type
Description
Camera Model
String
Camera information
Creation Date
Date
Image creation date
Software

String

Editing software used

GPS Data

String

Location information if available

Table 3.3 CNN Prediction

Field Name

Data Type

Description

Prediction

String

Real or Fake

Confidence

Float

Prediction confidence

ELA Status

String

Generated Successfully

ORB Result

String

Copy-Move Detection Result

Advantages of the Storage Design

No database installation required.

Lightweight architecture.

Faster execution.

Easy deployment.

Reduced storage requirements.

Simple maintenance.

Modular organization.

Easy future integration with cloud databases if required.

4. CODING

4.1. Sample Code:

```
import streamlit as st
import numpy as np
from PIL import Image
from tensorflow.keras.models import load_model
from ela import convert_to_ela
from orb import detect_copy_move

def check_metadata(uploaded_file):
    try:
        img = Image.open(uploaded_file)
        exif_data = img._getexif()
        uploaded_file.seek(0)

        if not exif_data:
            return "No Camera Metadata Found (Common in AI/Internet images)", False
        return "Camera Metadata Detected (Likely a Real Photo)", True
    except Exception:
        return "Metadata analysis failed", False

@st.cache_resource
def get_model():
    return load_model("model.h5")

st.title("Fake Image Detection System")

with st.sidebar:
    st.header("How to use")
    st.info("""
    1. Upload Original Files
    2. Avoid Screenshots
    3. Check ELA
    """)

model = get_model()

uploaded = st.file_uploader(
    "Upload an Image",
```

```

    type= ['jpg', 'jpeg', 'png']
)
if uploaded:
    image = Image.open(uploaded)
    st.image(image, caption="Uploaded Image")

    metadata_msg, metadata_found = check_metadata(uploaded)
    st.info(metadata_msg)

    ela_img_array = convert_to_ela(uploaded, target_size=(128,128))
    st.image(ela_img_array)

    processed = np.reshape(ela_img_array, (1,128,128,3))

    prediction = model.predict(processed)

    score = prediction [0][0]
orb_score = detect_copy_move(uploaded)

st.write(f"ORB Keypoint Matches: {orb_score}")

if orb_score > 50:
    st.warning("High number of matching keypoints detected.")
else:
    st.success("No significant Copy-Move patterns detected.")
if not metadata_found:
    if score < 0.05:
        st.error("Likely a FAKE / AI Generated Image")
    else:
        st.error("Likely a FAKE / TAMPERED Image")
else:
    if score < 0.5:
        st.success("Likely a REAL Image")
    else:
        st.error("Likely a FAKE Image")

```

ERROR LEVEL ANALYSIS (ela.py):

```
from PIL import Image, ImageChops, ImageEnhance
import numpy as np
```

```
def convert_to_ela(image_path, quality=90, target_size= (128,128)):
```

```
    original = Image.open(image_path). convert('RGB')
```

```
    temp_path = "temp_processing.jpg"
```

```
    original.save(temp_path, 'JPEG', quality=85)
```

```
    compressed = Image.open(temp_path)
```

```
    ela = ImageChops.difference(original, compressed)
```

```
    extrema = ela.getextrema()
```

```
    max_diff = max ([ex [1] for ex in extrema])
```

```
    if max_diff < 1:
```

```
        max_diff = 10
```

```
    scale = 255.0 / max_diff
```

```
    ela = ImageEnhance.Brightness(ela). enhance(scale)
```

```
    ela = ela.resize(target_size)
```

```
    return np.array(
```

ORB Detection(orb.py):

```
import cv2
import numpy as np

def detect_copy_move(image_path):

    image_path.seek(0)

    file_bytes = np.asarray(
        bytearray(image_path.read()),
        dtype=np. uint8
    )

    img = cv2.imdecode(file_bytes,0)

    if img is None:
        return 0

    orb = cv2.ORB_create ()

    kp, des = orb.detectAndCompute(img,None)

    if des is None:
        return 0

    bf = cv2.BFMatcher(
        cv2.NORM_HAMMING,
        crossCheck=True
    )

    matches = bf.match(des,des)

    real_matches = [m for m in matches if m.distance < 4]

    return len(real_matches)
```


5. TESTING

5.1. Introduction to Testing:

Software testing is an essential phase of the Software Development Life Cycle (SDLC) that ensures the developed software functions correctly, efficiently, and reliably. The primary objective of software testing is to identify defects, verify that the application satisfies the specified requirements, and ensure that the final product meets user expectations. Testing helps improve software quality by detecting errors before deployment, reducing maintenance costs, and increasing user confidence.

The Fake Image Detection System was tested thoroughly to verify the correctness of each module. Since the application integrates Artificial Intelligence, Image Processing, and Digital Image Forensics, testing was performed on individual modules as well as on the complete system. Every function including image uploading, metadata extraction, Error Level Analysis (ELA), ORB Copy-Move Detection, CNN prediction, and result generation was validated using different test images.

The application was tested with both genuine and manipulated images to evaluate prediction accuracy. Various image formats such as JPG, JPEG, and PNG were used to ensure compatibility. Invalid file formats and corrupted images were also tested to verify the robustness of the application. The testing process confirmed that the system correctly performs image preprocessing, executes all forensic analyses, generates predictions, and displays the final results through the Streamlit interface.

Software testing improves the reliability, usability, and maintainability of the Fake Image Detection System by ensuring that each component performs its intended function without errors.

5.2. Test Cases (Module-wise):

Test Case 1 – Image Upload Module

Test Case ID	TC-01
Module	Image Upload
Objective	Verify image upload functionality
Input	Valid JPG/JPEG/PNG image
Expected Result	Image uploaded successfully
Actual Result	Image uploaded successfully
Status	Pass

Test Case 2 – Invalid File Upload

Test Case ID	TC-02
Module	File Validation
Objective	Verify unsupported file rejection
Input	PDF / TXT File
Expected Result	Display error message
Actual Result	Unsupported file rejected
Status	Pass

Test Case 3 _ Metadata Analysis

Test Case ID	TC-03
Module	Metadata Analysis
Objective	Verify metadata extraction
Input	Original Camera Image
Expected Result	Camera metadata detected
Actual Result	Metadata displayed successfully
Status	Pass

Test Case 4 – Error Level Analysis (ELA)

Test Case ID	TC-04
Module	ELA Analysis
Objective	Generate ELA image
Input	Uploaded Image
Expected Result	Display ELA output
Actual Result	ELA image generated successfully
Status	Pass

Test Case 5 – ORB Copy-Move Detection

Test Case ID	TC-05
Module	ORB Detection
Objective	Detect duplicated image regions
Input	Copy-Move Forged Image
Expected Result	Detect high key point matches
Actual Result	ORB key points detected correctly
Status	Pass

Test Case 6 – CNN Prediction

Test Case ID	TC-06
Module	CNN Prediction
Objective	Classify image
Input	Genuine Image
Expected Result	Real Image
Actual Result	Real Image Predicted
Status	Pass

Test Case 7 – Fake Image Detection

Test Case ID	TC-07
Module	Final Prediction
Objective	Detect manipulated image
Input	Fake Image
Expected Result	Fake Image
Actual Result	Fake Image Predicted
Status	Pass

Test Case 8 – AI Generated Image Detection

Test Case ID	TC-08
Module	AI Detection
Objective	Detect AI-generated image
Input	AI Generated Image
Expected Result	Fake / AI Generated
Actual Result	AI Image Detected Successfully
Status	Pass

Test Case 9 – Result Display Module

Test Case ID	TC-09
Module	Result Display
Objective	Verify output display
Input	Any Valid Image
Expected Result	Display prediction with confidence score
Actual Result	Results displayed correctly
Status	Pass

Test Case 10 – Technical Analysis Module

Test Case ID	TC-10
Module	Technical Analysis
Objective	Display technical details
Input	Uploaded Image
Expected Result	Display metadata, confidence score, analysis technique
Actual Result	Technical details displayed correctly
Status	Pass

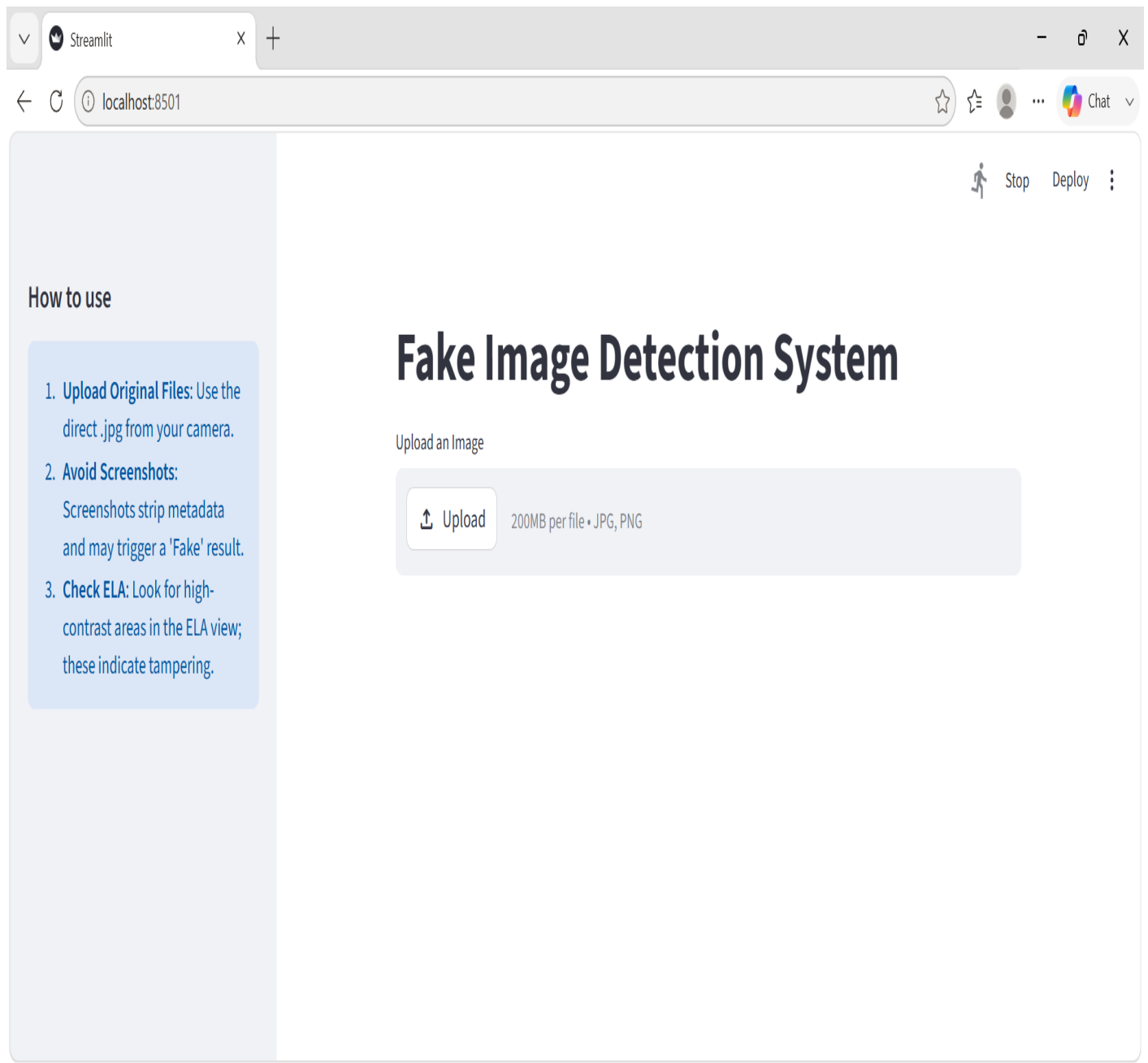
6. SCREENSHOTS

The following screenshots demonstrate the working of the Fake Image Detection System. They illustrate the user interface, image upload process, forensic analysis, prediction results, and technical details generated by the application.

6.1. Input Screens (Forms):

6.1.1 Home Page

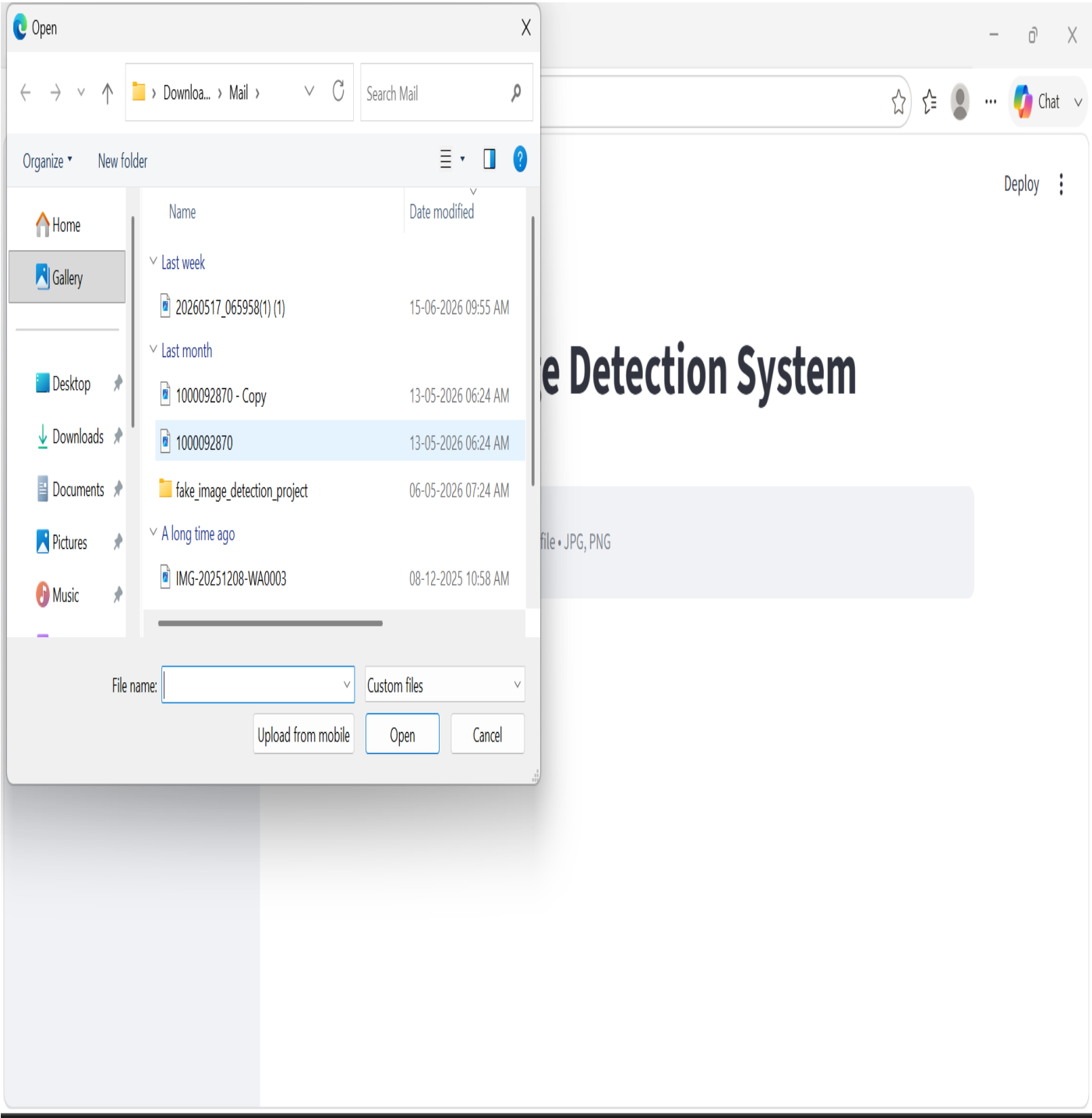
Figure 6.1: Home Page of Fake Image Detection System



Description: The home page displays the title of the application along with basic instructions and provides the interface for uploading an image.

6.1.2 Image Upload Screen

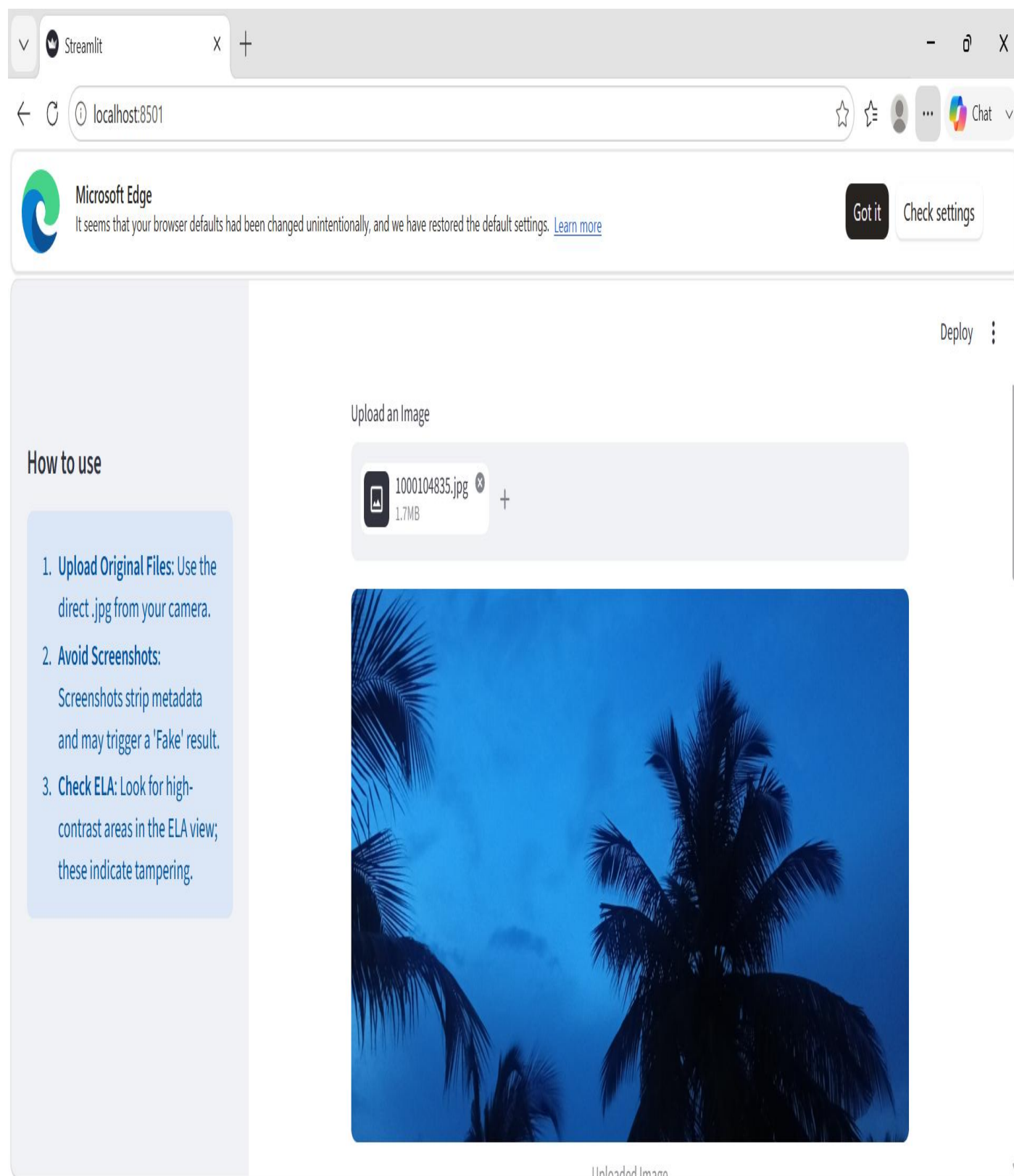
Figure 6.2: Image Upload Form



Description: This screen allows the user to upload an image in JPG, JPEG, or PNG format for fake image detection.

6.1.3 Uploaded Image Preview

Figure 6.3: Uploaded Image Preview

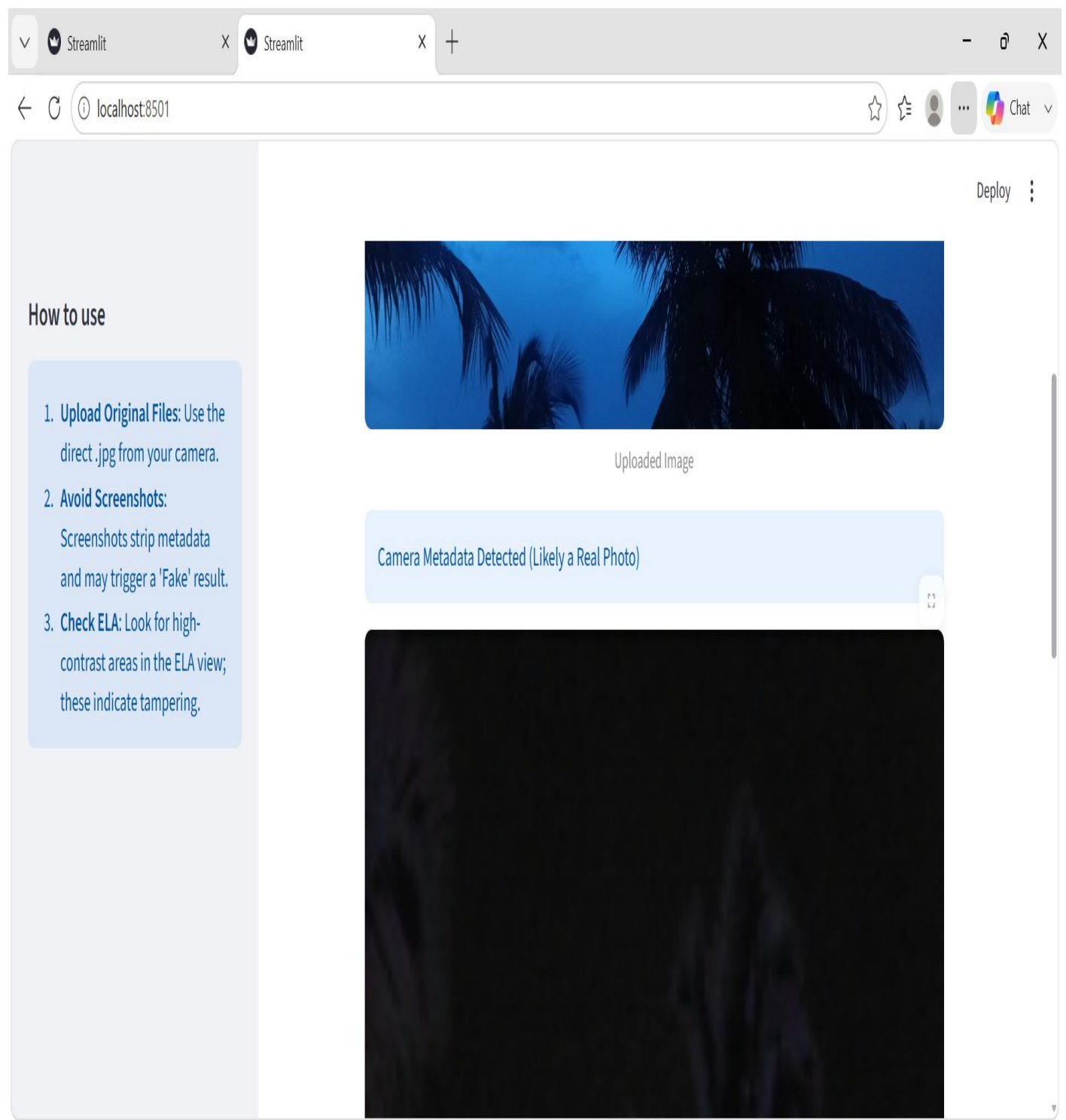


Description: After selecting an image, the application displays the uploaded image for user verification before performing analysis.

6.2 Output Screens (Reports)

6.2.1 Metadata Analysis Output

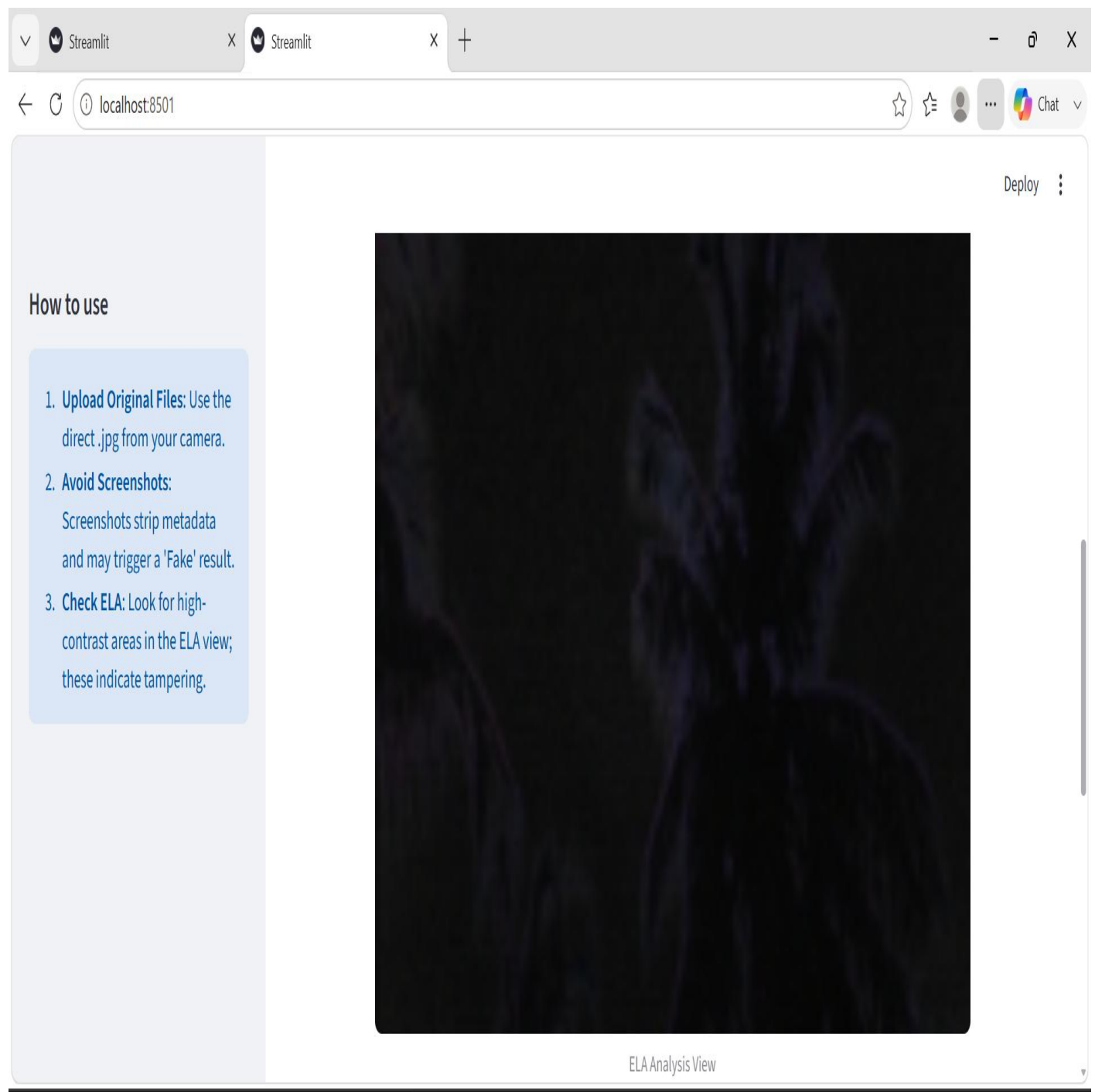
Figure 6.4: Metadata Analysis Result



Description: This screen displays whether camera metadata is detected or missing in the uploaded image.

6.2.2 Error Level Analysis (ELA) Output

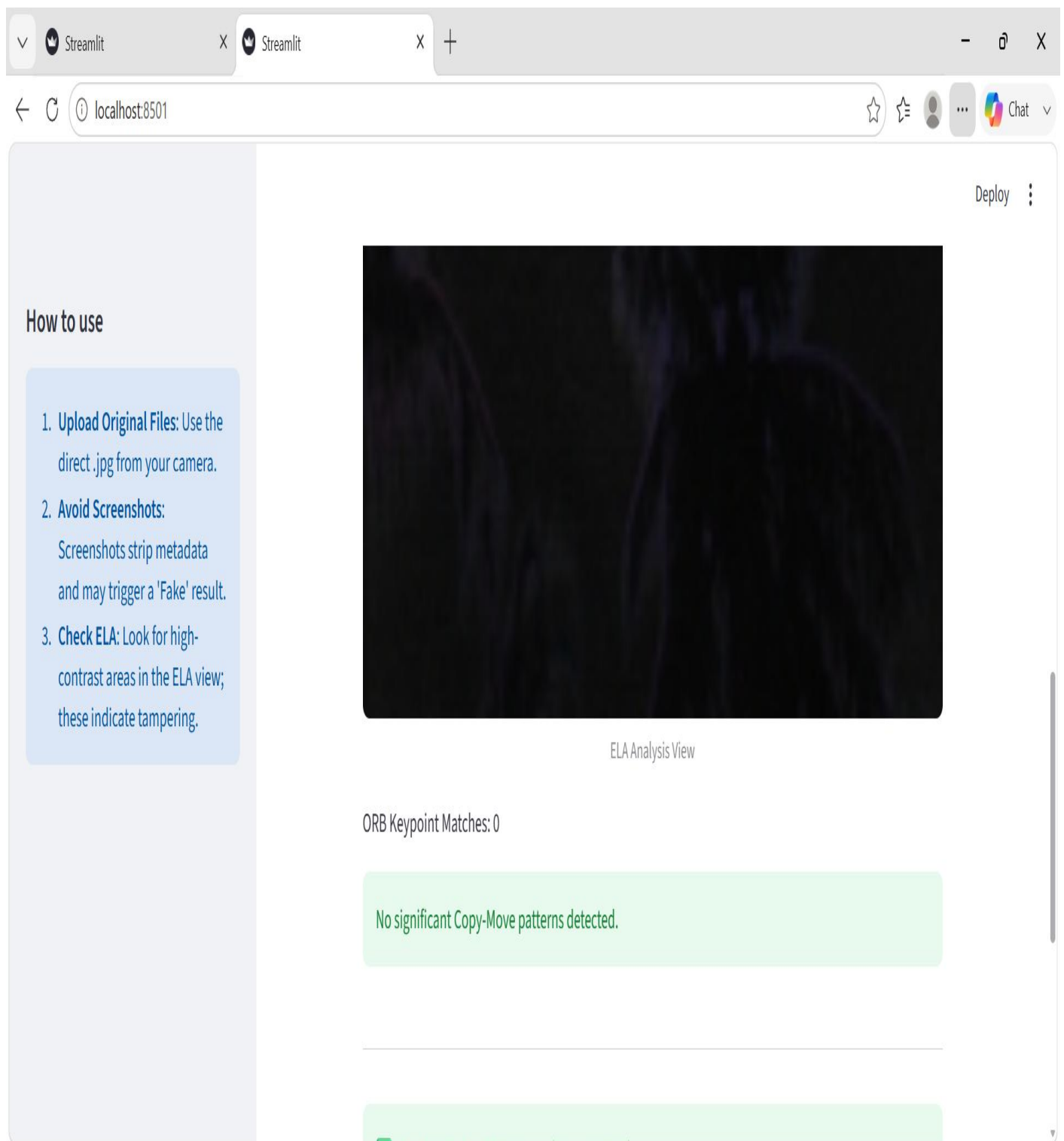
Figure 6.5: ELA Analysis Result



Description: The Error Level Analysis output highlights compression differences that may indicate image manipulation.

6.2.3 ORB Copy-Move Detection Output

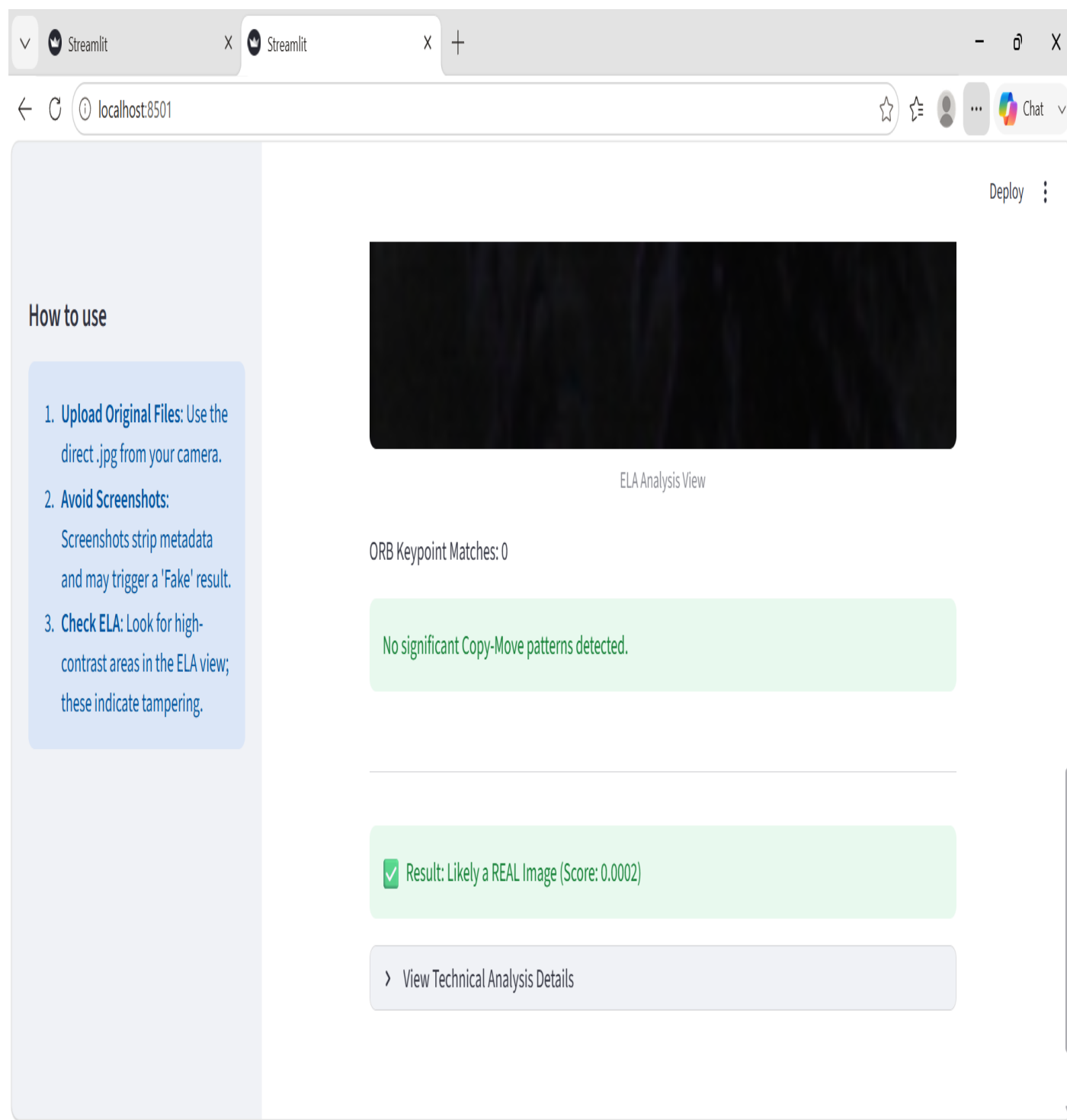
Figure 6.6: ORB Analysis Result



Description: This screen displays the ORB keypoint matching result used to detect copy-move forgery.

6.2.4 Final Prediction Output

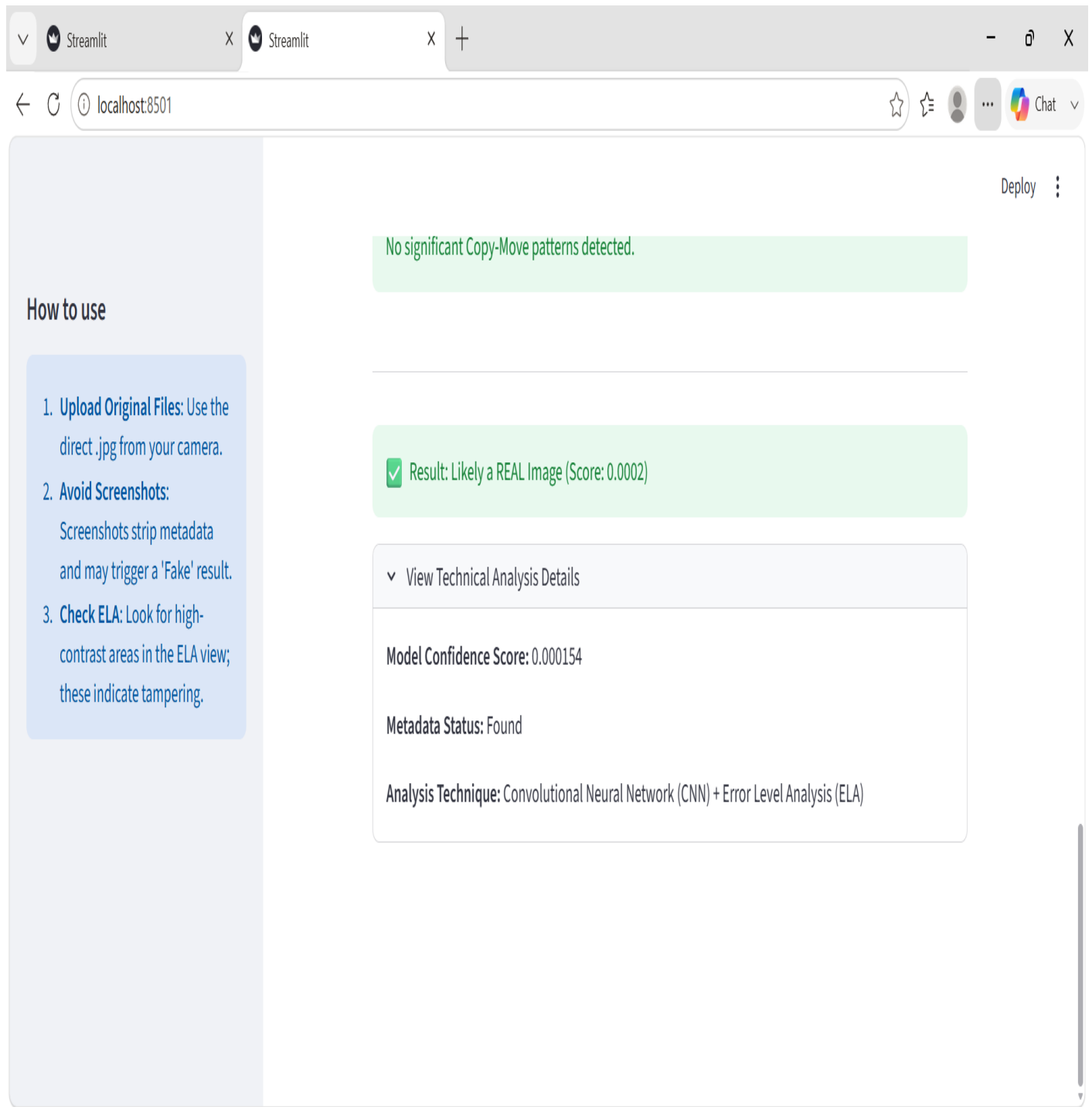
Figure 6.7: Fake Image Detection Result



Description: The application displays the final prediction indicating whether the uploaded image is Real or Fake, along with the confidence score.

6.2.5 Technical Analysis Output

Figure 6.8: Technical Analysis Details



Description: This screen displays additional information such as the model confidence score, metadata status, and the image analysis technique used for prediction.

7. CONCLUSION

The Fake Image Detection System was successfully designed, implemented, and tested to identify whether a digital image is genuine or manipulated using Artificial Intelligence and Digital Image Forensics techniques. The project combines Metadata Analysis, Error Level Analysis (ELA), ORB Copy-Move Detection, and a Convolutional Neural Network (CNN) to provide accurate and reliable image authentication. The application was developed using the Python programming language with the Stream lit framework, which provides a simple and interactive interface for uploading images and viewing analysis results. The system processes the uploaded image by extracting metadata, performing Error Level Analysis to detect compression inconsistencies, identifying duplicated regions using the ORB algorithm, and finally classifying the image as Real or Fake using the trained CNN model. The testing phase confirmed that each module performs its intended function correctly and that the application produces consistent and satisfactory results for different types of images, including genuine photographs, manipulated images, and AI-generated images. The modular architecture adopted in the project improves readability, maintainability, and scalability while allowing individual components to be upgraded independently. This project has provided valuable practical knowledge in the areas of machine learning, image processing, deep learning, and digital forensics, while also demonstrating how these technologies can be integrated into a real-world application. Overall, the objectives defined at the beginning of the project have been successfully achieved, and the developed system provides an efficient, user-friendly, and reliable solution for detecting fake digital images.

The future scope of the Fake Image Detection System is extensive because digital image manipulation techniques continue to evolve rapidly with advancements in Artificial Intelligence. The system can be enhanced by training the Convolutional Neural Network on larger and more diverse datasets to improve prediction accuracy and reduce false classifications. Advanced deep learning architectures such as Resnet, Efficient Net, or Vision Transformers (Vitt) can be incorporated to improve feature extraction and classification performance. Additional forensic techniques, including frequency-domain analysis, noise pattern analysis, lighting inconsistency detection, and GAN artifact detection, can further strengthen the system's capability to identify sophisticated image forgeries. The application can also be extended to detect deepfake images and manipulated videos, making it more useful in modern digital forensic investigations. Future versions may include cloud-based deployment, mobile application support, batch image analysis, real-time verification through APIs, and integration with social media platforms to automatically identify suspicious images before they are shared. Security can be enhanced by incorporating blockchain-based image authentication and digital watermark verification to ensure image integrity. With these improvements, the Fake Image Detection System can become a comprehensive and scalable forensic tool that can be effectively used in journalism, law enforcement, cybersecurity, legal investigations, healthcare, banking, government organizations, and educational institutions, helping to combat misinformation and improve trust in digital media.

8. BIBLIOGRAPHY

Ian Goodfellow, Yoshua Bengio, and Aaron Courville, Deep Learning, MIT Press, 2016.

Richard Szeliski, Computer Vision: Algorithms and Applications, Springer, 2nd Edition, 2022.

Rafael C. Gonzalez and Richard E. Woods, Digital Image Processing, Pearson Education, 4th Edition, 2018.

Simon J.D. Prince, Computer Vision: Models, Learning, and Inference, Cambridge University Press, 2012.

TensorFlow Developers, TensorFlow Documentation, Available at: <https://www.tensorflow.org>

Keras Team, Keras Documentation, Available at: <https://keras.io>

OpenCV Team, OpenCV Documentation, Available at: <https://opencv.org>

Streamlit Developers, Stream lit Documentation, Available at: <https://streamlit.io>

Python Software Foundation, Python Documentation, Available at: <https://docs.python.org/3>

Pillow Developers, Pillow (PIL) Documentation, Available at: <https://pillow.readthedocs.io>

NumPy Developers, NumPy Documentation, Available at: <https://numpy.org>

David Lowe, Distinctive Image Features from Scale-Invariant Key points, International Journal of Computer Vision, 2004.

Ethan Rublee, Vincent Rabaud, Kurt Konolige, and Gary Bradski, ORB: An Efficient Alternative to SIFT or SURF, International Conference on Computer Vision (ICCV), 2011.

Hany Farid, Image Forgery Detection, IEEE Signal Processing Magazine, 2009.

M. Stamm, M. Wu, and K. J. R. Liu, Information Forensics: An Overview of the First Decade, IEEE Access, 2013.